



NOWITNESS

Security Audit Report

Cardano Programmable Tokens Standard

Date	August 10, 2026
Client	Cardano Foundation
Project	CIP-113 Programmable Tokens
Version	0.3.0

Contents

Disclosure	1
Disclaimer	2
Scope	4
Assessment Overview	5
Assessment Components	6
Security Assessment Framework	7
Executive Summary	8
Code Base	12
Severity Classification	14
Finding Severity Ratings	15
Findings	16
[C 501] Escape of Minted Tokens during Third-Party Action	18
[C 502] ThirdPartyAct UTXO Contamination	22
[H 401] Reference Script Preservation in ThirdPartyAct Path	31
[M 301] Enforcement of transfer_logic_script for Purely Minted Tokens	34
[M 302] Composability Problem in TransferAct Path	37
[M 303] Unconstrained Asset Addition in Smart-Wallets	40
[L 201] Registry Init Does Not Bind Origin Node to registry_spend	42
[L 202] Registration and Transfer Coupling Issue	45
[I 101] Ambiguity in the Scope of Programmable Control	49
[I 102] Linked List Registry Limitation	52
[I 103] Separation of Concerns	55
[I 104] Inefficient Membership Check in issuance_mint Validator	58
[I 105] Indirect Delegation Signal in PLB-Input Scan	60
[I 106] Redundant single_mint_with_credential Check in issuance_mint	62
[I 107] Redundant Checks and Misleading Variable Names	65
[I 108] ThirdPartyAct Cannot Atomically Operate Across Multiple Token Policies ..	70
[I 109] Token UTXO Fragmentation Causes Non-Atomic Seizure	72
[I 110] ThirdPartyAct Scope is Not Clearly Defined	74
[I 111] Registry Design Limitation	76
Appendix A: Supporting Validation Matrices	80

Disclosure

This document contains proprietary information belonging to No Witness Labs. Duplication, redistribution, or use, in whole or in part, in any form, requires prior written consent from No Witness Labs.

Nonetheless, both the customer **Cardano Foundation** and No Witness Labs are authorized to share this document with the public to demonstrate security compliance and transparency regarding the outcomes of the Protocol.

Disclaimer

This review reflects the state of the codebase at the time of assessment and should be regarded as a point-in-time evaluation. Any changes made after the assessment period fall outside the scope of this report.

Although every effort was made to identify potential risks and vulnerabilities, no security review can guarantee the discovery of all issues. The information and recommendations in this report are provided solely for technical security purposes and must not be considered legal, regulatory, financial, or investment advice.

To strengthen security assurance, projects are encouraged to seek multiple independent reviews and to supplement audits with practices such as continued testing, monitoring, formal verification where appropriate, and bug bounty programs.

The scope of this audit was limited to the code and materials provided for review. It did not extend to compiler-level artefacts, generated UPLC code, deployment infrastructure, off-chain transaction construction, user interfaces, wallets, dApps, indexers, backend services, or areas beyond the explicitly reviewed codebase. Developing a comprehensive unit-test suite, property-based test suite, or formal verification model was also outside the scope of this engagement.

Per-Policy Script Dependency

CIP-113 intentionally delegates token-specific behavior to per-policy scripts and substandards, including each token's `transfer_logic_script`, `third_party_transfer_logic_script`, and `issuance_logic_script`. These scripts define the rules for user transfers, third-party administrative actions, and minting or burning authorization for a given programmable token.

This review assessed the shared CIP-113 framework and reference validators, but did not provide assurance over arbitrary issuer-defined or third-party-deployed per-policy logic. As a result, the security of any particular programmable token also depends on the correctness, integrity, and administrative assumptions of the per-policy scripts selected by its creator or issuer.

Users, integrators, and stakeholders should independently review the registered per-policy scripts and any associated substandard before relying on or integrating a programmable token.

Administrative and Governance Assumptions

CIP-113 supports token-specific third-party administrative behavior through registered per-policy logic. This report assesses technical behavior within the reviewed scope, but does not evaluate whether a given administrative model, issuer policy, governance process, freeze/seizure mechanism, or compliance workflow is legally, commercially, or operationally appropriate. Users and integrators should understand the administrative authority granted by each programmable token before relying on it.

Trusted Reference UTxO and Configuration Assumptions

The security properties described in this report assume that all protocol-level reference UTxOs and configuration artefacts are correctly initialized, authentic, and used as intended. This includes, but is not limited to, the protocol parameters UTxO, registry-related configuration, issuance script template data, script hashes, minting policy identifiers, bootstrap artefacts, and any fields read from referenced UTxOs, such as issuance-policy prefix and postfix bytes.

These referenced values were treated as trusted configuration inputs for the purpose of this review. The assessment focused on the behavior of the reviewed validators under the assumption that these values, their identifying tokens or policies, and their on-chain locations correspond to the intended canonical CIP-113 deployment. Incorrect initialization, malformed reference data, use of unintended reference UTxOs, non-canonical parameters, or deployment under different configuration assumptions may invalidate conclusions made in this report.

Scope

In scope

- On-chain validators and supporting logic at the time of review.
- Datum/redeemer structures and on-chain invariants (value, units/rates, cross-phase checks).
- Use of reference UTxOs for configuration and script parameters.

Out of scope

- Compiler layer / UPLC output and toolchain internals.
- Creation of new unit tests or property-based tests.
- Off-chain libraries, front-end apps, wallet UX (incl. blind-signing), unless explicitly constrained on-chain.
- Third-party protocol internals (bridges/liquidity venues), centralized services, exchanges, oracles, external price feeds.
- Node operations, networking, mempool policy variance, ordering/MEV-like effects.

Assessment Overview

From **April 23, 2026** to **June 01, 2026**, **Cardano Foundation** engaged No Witness Labs to conduct a comprehensive security assessment of the **CIP-113 Programmable Tokens** protocol. The evaluation was performed in accordance with established industry best practices for code auditing and protocol analysis.

The audit followed a structured five-phase process:

- **Planning** – Defined the assessment scope and objectives, and aligned expectations with the client.
- **Discovery** – Acquired an understanding of the protocol through client discussions, documentation review, and preliminary analysis of architecture and design.
- **Manual Review** – Conducted an in-depth review of the codebase to identify potential vulnerabilities, design weaknesses, and security risks.
- **Validation** – Verified client-provided fixes and obtained acknowledgments for resolved issues.
- **Reporting** – Documented all confirmed findings, assigned severity levels, and provided actionable recommendations for remediation.

The team also evaluated protocol efficiency and potential optimizations. Findings were prioritized by severity level.

Assessment Components

Manual Review

Our manual code auditing is focused on a wide range of attack vectors, including but not limited to:

- Business logic and protocol-specific invariants
- UTXO Value Size Spam (Token Dust Attack)
- Large Datum or Unbounded Protocol Datum
- EUTXO Concurrency DoS
- Cheap spam
- Unauthorized Data modification
- Multisig PK Attack
- Infinite Mint
- Incorrect Parameterized Scripts
- Other Redeemer
- Other Token Name
- Script output datum
- Arbitrary UTXO Datum
- Unbounded protocol value
- Foreign UTXO tokens
- Double or Multiple satisfaction
- Locked Ada
- Locked non Ada values
- Missing UTXO authentication
- Stake-credential ownership and authorisation controls
- Value and accounting calculations
- Integer sign and token-quantity validation
- Minimum Ada requirements
- UTXO contention

Security Assessment Framework

This assessment combined four complementary review layers. No single checklist was treated as exhaustive.

- **No Witness Labs audit baseline** – The review used the internal security baseline maintained by No Witness Labs, covering common vulnerability categories and audit checks. The principal categories applicable to this engagement are presented in the Assessment Components section.
- **Applicable Cardano Improvement Proposals (CIPs)** – CIPs applicable to the protocol's reviewed functionality were identified, and their relevant implementation requirements were assessed for correctness and security. Unrelated CIPs were not applied indiscriminately.
- **Protocol specification and security invariants** – The implementation was evaluated against the protocol's intended design, supplied documentation, and stated invariants, including authorisation rules, asset preservation, state transitions, accounting logic where applicable, and other protocol-specific security properties.
- **Adversarial and economic analysis** – The review considered unexpected or attacker-controlled transaction constructions, authorisation bypasses, value extraction, state manipulation, denial-of-service conditions, economic abuse, and edge cases arising from interactions among validators and other protocol components.

Executive Summary

Project Overview

CIP-113 (Programmable Tokens) is a Cardano Improvement Proposal that defines a standard for programmable Cardano native tokens whose ownership changes require successful script execution. The standard introduces a shared smart-wallet architecture in which programmable tokens are held at addresses using `programmable_logic_base` as the payment credential, while ownership is identified by each address's stake credential.

The `programmable_logic_base` validator acts as a lightweight gatekeeper that delegates validation to `programmable_logic_global`, which verifies registry membership and ensures that the relevant per-policy transfer or third-party logic is executed before programmable tokens can be transferred or administratively acted upon.

Extended Overview

Design Overview

CIP-113 introduces a shared programmable-token framework consisting of three core on-chain components:

- `programmable_logic_base` (PLB): A shared spend script used as the fixed payment credential for programmable-token smart-wallet addresses. Programmable tokens are held in UTXOs at addresses using this payment credential, while ownership is identified by each address's stake credential.
- `programmable_logic_global` (PLGlobal): A withdraw-zero stake validator that performs the main validation for transfers and third-party actions involving PLB UTXOs. It verifies registry proofs and requires the relevant per-policy transfer or third-party logic script to be executed.
- Registry (linked list): An on-chain sorted linked list of `RegistryNode` UTXOs. Each node represents one registered programmable-token policy and records the policy ID, the `transfer_logic_script` credential, the `third_party_transfer_logic_script` credential, and an optional global-state currency symbol.

Key Characteristics

- Token-specific transfer rules are defined per policy through a `transfer_logic_script` credential. For `TransferAct` transactions, each registered token policy present in spent PLB inputs must be proven through the registry and must have its corresponding transfer logic executed.
- Third-party administrative actions are handled through a `ThirdPartyAct` path that invokes the targeted policy's `third_party_transfer_logic_script`. This path can support actions such as seizure, forced transfer, wipe, top-up, or other policy-specific administrative behavior defined by the token substandard. In the audited implementation, each `ThirdPartyAct` targets a single registered policy, although it may process multiple PLB UTXOs for that policy.

- Registry membership and non-membership are proven through ordered linked-list inclusion and exclusion proofs. Inclusion is proven by referencing the `RegistryNode` whose `key` matches the token policy, while non-membership is proven by referencing the covering node whose `key` is lower than the policy and whose `next` is greater than the policy.

Roles and Responsibilities

- Token Creators: Create a programmable token policy, define the token-specific `transfer_logic_script`, `third_party_transfer_logic_script`, and `issuance_logic_script`, and register the policy in the registry.
- Token Holders: Hold programmable tokens in smart-wallet UTXOs that use `programmable_logic_base` as the payment credential and the holder's credential as the stake credential. Holders initiate `TransferAct` transactions that satisfy the relevant registry proofs and transfer logic scripts.
- Token Issuers: Mint or burn programmable tokens according to the token-specific `issuance_logic_script` and `issuance_mint` policy.
- Third-Party Operators / Admins: Invoke `ThirdPartyAct` to perform policy-specific administrative actions, such as seizure, forced transfer, wipe, or migration, when authorized by the registered `third_party_transfer_logic_script`. Any minting or burning involved in such actions must also satisfy the token's issuance policy.

Lifecycle of Assets

- Registration: A token policy is registered by spending the covering registry node, inserting a new `RegistryNode` into the ordered linked list, updating the predecessor node, and minting the corresponding registry-node NFT. Token issuance is handled separately by the issuance minting policy.
- Transfer: Token holders spend PLB UTXOs via `TransferAct`, which requires registry membership or non-membership proofs for each distinct token policy involved in the spent PLB inputs. For each registered policy, the corresponding `transfer_logic_script` must be invoked.
- Third-Party Action: A third-party operator invokes `ThirdPartyAct` to perform a policy-specific administrative action, subject to the `third_party_transfer_logic_script` of the targeted policy. Any minting or burning performed as part of the action must also satisfy the token's issuance policy.

- Burning: Programmable tokens are burned through the issuance minting policy with the required token-specific authorization and, when PLB UTxOs are spent, the corresponding PLGlobal validation path.

Supporting Infrastructure

The reference implementation is written in Aiken and targets Cardano's eUTxO model. The linked-list registry supports both inclusion proofs (`TokenExists`) and exclusion proofs (`TokenDoesNotExist`), allowing PLGlobal to distinguish registered programmable-token policies from unregistered token policies without scanning the full registry on-chain.

Risks and Dependencies

The security of the programmable-token framework depends on the correct initialization and continued integrity of the registry linked list, the correctness of the shared PLB/PLGlobal validation path, and the soundness of each token-specific `transfer_logic_script`, `third_party_transfer_logic_script`, and `issuance_logic_script`. It also depends on correct cross-validator composition between `programmable_logic_base`, PLGlobal, `issuance_mint`, and the registry validators.

Key risks identified during this audit relate to bootstrap and registry trust assumptions, composability of multi-policy PLB UTxOs, mint/burn and transfer-path interactions, and the scope of administrative authority granted through third-party operator logic.

Code Base

Repository

<https://github.com/cardano-foundation/cip113-programmable-tokens>

Branch

main

Audited Commit

31204e9ddf5840ef8d6991d0c37108df31bb1cd0

Fixes Verified Commit

f16c85794345d36288de4802cfc29dc74668b7d

Files Audited

Files	SHA256 Checksum
validators/always_fail.ak	3388ade147c1349a7dcf1ca2b045d7ff159a8e8fa01550c52fc8874b8631106a
validators/issuance_cbor_hex_mint.ak	38ece2d3332c5a0d261ec60b4604b5eea51a3effd9117449b4ef6541784ff322
validators/issuance_mint.ak	0fc45c31fcf526955a80a3fc786e1fe2153a7249929aa421614e8c01654c6aec
validators/programmable_logic/benchmark.aks	18eb2dddfec9e0ac46bc93806f05210cf5161e2f28ed5743b7e3931cd393e5bf
validators/programmable_logic/fixture.ak	1f660b8bd3f7cd970d64b6633bf16267053b69bca154bb85a49978b0506ab8c0
validators/programmable_logic/params.ak	e6e071a40941e384084115069ef146e7142079963b84d18d77caf0b0f9f9d166
validators/programmable_logic/third_party.ak	ec87da05503c1725999c6593f05c29f71bf036cc6dc79c44abef70968c1040e1
validators/programmable_logic/transfer.ak	51425dcbaddaefb883473f764a21ad8ba33053d28b0b2aacd1eca63cda46da67

Files	SHA256 Checksum
validators/programmable_logic_base.ak	0b1823f19927f37703cfb06db0f60ad0964e157d27be53bae3980b74207e58a6
validators/programmable_logic_global.ak	27ce59dea618ee747e95f7bf8c1c5080f22afcd205102640339847ecbf24b287
validators/protocol_params_mint.ak	41397dada958ff1de052884288832d44edb84599e305ff92f020e1f077e7fd0c
validators/registry_mint.ak	5c38f737d4d9a3714414f8520d64b5c113f247c77293585fe3a1f329afbd1ce0
validators/registry_spend.ak	140c60ec4ec603665aada1b04b6d2fe9d6020afa8f26f6a8481c1d8e6f9d7013
lib/assets.ak	0c6b92d38d4771c5d2955dfe272b17bbc08c4361d9581b789cf15b9af14db261
lib/linked_list.ak	85c9b2795f5790b7123139f59dcfb808adaf422014a2423cdd1a25eb453c0a60
lib/list.ak	01c1bd920f0f18769a09e8dddb2cd493bd43e2dddec3df8c00749d3e777435d7
lib/pairs.ak	f61ffb48c246c24c3fa15e130677eee205dfecaba93644e8b017369b01b52175
lib/registry_node.ak	b966e351d41178a4bb3506156fdcba559918bfbcb02b8f051ab7fa3e70fe03708
lib/tokens.ak	57dbf4a4a9693b7f55c71ccbfaeb60238f50a3ff567905d31c7156ab4ca482ab
lib/types.ak	d76a26b3bde84892213a6aaab51076b208ad897deb584e04ffff28514ffff51912
lib/utls.ak	74f1c3c2519aafb0d8a3aa9a80aff3fbd926cf3eefb9ebe5d20880b42dbbc023

Severity Classification

- **Critical:** Vulnerabilities that enable direct theft of assets, permanent loss of funds, or catastrophic protocol failure. These represent immediate and severe financial risk with straightforward exploitation paths.
- **High:** Vulnerabilities that can result in financial loss or significant harm to users or the protocol, though impact may be limited to specific functions or require certain conditions. Exploitation paths exist but may be more complex than Critical Vulnerabilities.
- **Medium:** Vulnerabilities that do not directly result in financial loss but may temporarily impair functionality or create exploitable conditions. These are typically limited to state manipulations and include issues such as front-running susceptibility or logic flaws that compromise transaction integrity.
- **Low:** Low-impact Vulnerabilities with limited attack vectors, minimal attacker incentive, or requiring highly specific scenarios. These typically do not result in financial loss or significant harm to users or the protocol.
- **Informational:** Findings without immediate security or financial implications. These include optimization opportunities, code quality improvements, inconsistencies in naming conventions, design suggestions, and gaps in documentation.

Impact–Likelihood Severity Matrix

	High likelihood	Medium likelihood	Low likelihood
High impact	C	H / M	M
Medium impact	H	M	M / L
Low impact	M	M / L	L

Finding Severity Ratings

The following table defines levels of severity and score range that are used throughout the document to assess vulnerability and risk impact

	Level	Severity	Count
	5	Critical	2
	4	High	1
	3	Medium	3
	2	Low	2
	1	Informational	11
Total Findings			19

Findings

ID	Title	Severity	Status
501	Escape of Minted Tokens during Third-Party Action	Critical	Resolved
502	ThirdPartyAct UTxO Contamination	Critical	Resolved
401	Reference Script Preservation in ThirdPartyAct Path	High	Resolved
301	Enforcement of transfer_logic_script for Purely Minted Tokens	Medium	Resolved
302	Composability Problem in TransferAct Path	Medium	Mitigated
303	Unconstrained Asset Addition in Smart-Wallets	Medium	Mitigated
201	Registry Init Does Not Bind Origin Node to registry_spend	Low	Resolved
202	Registration and Transfer Coupling Issue	Low	Resolved
101	Ambiguity in the Scope of Programmable Control	Informational	Resolved
102	Linked List Registry Limitation	Informational	Acknowledged
103	Separation of Concerns	Informational	Resolved
104	Inefficient Membership Check in issuance_mint Validator	Informational	Resolved
105	Indirect Delegation Signal in PLB-Input Scan	Informational	Resolved
106	Redundant single_mint_with_credential Check in issuance_mint	Informational	Resolved

ID	Title	Severity	Status
107	Redundant Checks and Misleading Variable Names	Informational	Resolved
108	ThirdPartyAct Cannot Atomically Operate Across Multiple Token Policies	Informational	Acknowledged
109	Token UTxO Fragmentation Causes Non-Atomic Seizure	Informational	Resolved
110	ThirdPartyAct Scope is Not Clearly Defined	Informational	Partially Resolved
111	Registry Design Limitation	Informational	Partially Resolved

[C 501] Escape of Minted Tokens during Third-Party Action

	Level	Severity	Status
	5	Critical	Resolved

Category: Programmable Token Escape / Compliance Bypass

Affected Files:

- `validators/issuance_mint.ak`
- `validators/programmable_logic/third_party.ak`
- `validators/programmable_logic_global.test.ak`

Summary

`issuance_mint` delegates mint-output custody checks whenever the transaction spends any UTxO from `programmable_logic_base`.

That assumption is valid for mint-aware transfer flows, but it is not valid for `ThirdPartyAct`. The third-party path only enforces balance preservation for the seized policy referenced by its registry node.

As a result, a transaction can:

1. spend programmable token `A` under `ThirdPartyAct`
2. mint registered token `B` in the same transaction
3. route newly minted `B` to an arbitrary non-PLB address

All relevant validators accept the transaction:

- `issuance_mint`
- `programmable_logic_base`
- `programmable_logic_global`

Root cause

In the `RefInput` branch, `issuance_mint` decides whether to skip its own output-custody enforcement using this condition:

- ***Does the transaction spend any input whose payment credential is `programmable_logic_base` ?***

If yes, it assumes PLGlobal will validate the rest:

```

1 or {
2   list.any(
3     self.inputs,
4     fn(input) {
5       input.output.address.payment_credential == programmable_logic_base
6     },
7   ),
8   validate_mint_outputs(...)?,

```



```
9 }
```

While PLGlobal's `transfer_act` path does enforce custody for all programmable tokens in the transaction, PLGlobal's `third_party_act` path does not.

PLGlobal's `validate_3rd_party` function extracts a single seized policy from the registry node and only adjusts accounting for that policy:

```
1 let
2   policy_id,
3   third_party_logic,
4   <- with_key_and_3rd_party_logic(registry_node)
5
6 let minted_or_burned = self.mint |> value.tokens(policy_id)
```

 Aiken

This `minted_or_burned` does not include any other programmable tokens minted in the transaction, so PLGlobal does not enforce that they remain in programmable custody.

So when token `B` is minted while `ThirdPartyAct` is operating on token `A`, no core validator enforces that minted `B` remains at programmable custody.

`check_seized_tokens` does not close this gap. The `split_at` equality checks (`input_tokens_after == output_tokens_after`) only compare the seized input against its directly paired output — token `B` is not present in any input (it is freshly minted) so it never appears in `input_tokens_after`. The final output sweep filters by `output.address.payment_credential == prog_logic_cred`, which drops the non-PLB output carrying token `B`. Even if that filter were absent, `value.tokens(policy_id)` scopes to token `A`'s policy and returns empty for any token `B` output. Token `B` contributes nothing to any accumulator in the function.

Exploit Scenario

- Preconditions:
 - Token `B` is a registered programmable token
 - The attacker has valid mint authority for token `B`
- Attacker setup:
 - The attacker creates and registers a separate dummy programmable token `A` whose `third_party_transfer_logic_script` can be satisfied by the attacker.
 - The attacker then mints a small amount of token `A` into `programmable_logic_base`.
- Exploit transaction: The attacker builds a transaction that:
 - Spends the dummy token `A` UTxO from `programmable_logic_base`
 - Invokes `ProgrammableLogicGlobal` with `ThirdPartyAct` for token `A`
 - Executes token `A`'s `third_party_transfer_logic_script`

- Mints token `B` through `issuance_mint` using the `RefInput` registry proof path
- Sends the newly minted token `B` to a normal non-PLB address.
- Validation outcome:
 - `issuance_mint(B)` observes that the transaction spends a PLB input and delegates custody validation to `PLGlobal`
 - `PLGlobal` executes the `ThirdPartyAct` path for token `A`
 - `validate_3rd_party(A)` only accounts for token `A`'s policy
 - Minted token `B` is never checked for programmable custody
 - Token `B` is accepted at a non-PLB output, escaping the programmable shell.

Impact

This breaks the core invariant that registered programmable tokens minted through `issuance_mint` must remain under programmable custody.

A minter with valid mint authority for token `B` can create an unrelated dummy token `A` and use a self-authorized `ThirdPartyAct` on `A` as a false delegation trigger to bypass programmable custody checks for `B`. The attacker can then route minted `B` to a normal address, escaping the programmable shell.

For regulated assets, this is a programmable-custody escape and compliance bypass.

Likelihood rationale (High): creating a separate dummy programmable token is permissionless for any actor who can satisfy the standard registration requirements. Therefore, an authorized minter of token `B` does not need collusion with an unrelated third-party operator. They can first create and fund their own dummy token `A` with permissive third-party logic, then use a `ThirdPartyAct` on `A` as the PLB-spend trigger that causes `issuance_mint(B)` to delegate custody checks. The only remaining privilege required is valid mint authority for `B`.

Proof of exploit

Added a cross-validator exploit test:

- `validators/programmable_logic_global.test.ak`

Test name:

- `third_party_act_on_token_a_plus_issuance_mint_of_token_b_escape_succeeds`

The test:

- spends token `A` from PLB
- invokes `ThirdPartyAct` for token `A`
- mints token `B`
- routes minted token `B` to a non-PLB wallet output
- calls both `issuance_mint.mint(...)` and `programmable_logic_global.withdraw(...)`

The exploit shape is accepted by both validator calls.

Recommended remediation

Option 1

Modify `issuance_mint`'s delegation condition to check the PLGlobal redeemer:

- Only allow `issuance_mint` to delegate its output-custody checks when the PLGlobal's redeemer is `TransferAct`.
- Otherwise, if the redeemer is `ThirdPartyAct`, keep `issuance_mint`'s output-custody checks active.

Option 2

Modify `ThirdPartyAct` to enforce programmable-custody/accounting for every programmable policy minted in the transaction, not just the seized policy referenced by its registry node:

- Extend `ThirdPartyAct` to enforce programmable-custody/accounting for every programmable policy minted in the transaction.

Short Conclusion

This is a real cross-validator composition flaw:

- `issuance_mint` trusts PLGlobal too broadly
- `ThirdPartyAct` does not uphold that trust for other minted policies

The result is that a mint of token `B` can escape programmable custody when combined with a third-party action on token `A`.

Resolution

Issuance custody delegation now requires the PLGlobal withdrawal redeemer to cover the same verified registry node as the minted policy. An unrelated `ThirdPartyAct` therefore falls back to policy-wide, output-wide `no_escape` validation, closing the cross-policy mint escape.

[C 502] ThirdPartyAct UTxO Contamination

	Level	Severity	Status
	5	Critical	Resolved

Category: Authorization Scope Violation / UTxO Contamination / Denial of Service

Affected Files:

- `validators/programmable_logic_global.ak`
- `validators/programmable_logic/third_party.ak`

Description

In the `ThirdPartyAct` path of the `programmable_logic_global` validator, the `check_seized_tokens` function enforces that the tokens affected by the 3rd-party action (referred to as “seized tokens”) must change, but only those tokens. This means that all non-target-policy assets must remain unchanged, and only the target policy tokens can be modified.

However, the wording “Seized tokens” is misleading, because technically, the logic allows changing these tokens’ amounts in various ways, other than just seizing them (e.g. adding more tokens to existing tokens, or adding new tokens that were not present before).

For example, the 3rd-party action can be used to add new tokens (i.e. inject new tokens into the `value` of the output UTxOs). As long as the total amount of the target policy tokens is preserved across inputs (including minting or burning effects) and outputs, the transaction can still be valid. So, the 3rd-party action can be used to manipulate tokens in various ways, not just seizing them, which can lead to a wide range of potential attack vectors if not properly understood and mitigated.

Moreover, the `ThirdPartyAct` path can include any UTxO from the CIP-113's PLB address as input in the transaction. There is no restriction on which UTxOs can be included in the transaction.

Thus, a malicious programmable token’s `ThirdPartyAct` can contaminate arbitrary PLB UTxOs that did not previously contain that token, because the paired input is not required to already hold the acted-on policy.

Relevant Code Snippet

```

1  /// Analyze inputs and outputs affected by the 3rd-party action (grossly named
   *seized_tokens* here).
2  fn check_seized_tokens(..) -> Bool {
3      ...
4      // Seized tokens must change, but only them.
5      expect and {
6          (input_tokens_before == output_tokens_before)?,
7          (input_tokens_after == output_tokens_after)?,
8          (input_tokens_at != output_tokens_at)?,
```

 Aiken

```
9      }
10     ...
11 }
```

Here `input_tokens_at` and `output_tokens_at` refer to the amounts of the target policy tokens in the inputs and outputs affected by the 3rd-party action, while `input_tokens_before`, `output_tokens_before`, `input_tokens_after`, and `output_tokens_after` refer to the amounts of non-target-policy assets in the inputs and outputs before and after the 3rd-party action.

This means that `output_tokens_at` can be any value. This would allow the 3rd-party action not only to seize tokens, but also to add tokens to the `value` of the output UTxO. This means that the 3rd-party action can be used to inject new tokens into user's smart-wallet UTxOs.

Impact

1- DOS Attack

Since the `ThirdPartyAct` path allows including any UTxO from the smart-wallet as input, an attacker can potentially create a denial-of-service (DOS) attack by including the victim's UTxOs in their transaction, potentially blocking the victim from using those UTxOs.

This can be used to DOS the following:

- A user smart-wallet
- A DEX pool
- A lending pool
- Or block certain financial operations (e.g. by including the collateral UTxO of a DeFi position, the attacker can prevent liquidation, margin call, or closing a position during critical moments).
- Or can authorize some actions that depend on the presence of NFTs (e.g. Bond NFT) without the user's consent.
 - example: Some DeFi protocols authorize certain actions (e.g. claiming rewards, liquidating a position, etc.) by checking the presence of bond NFTs within the inputs of the transaction. An attacker can use the `ThirdPartyAct` to include the victim's UTxO that contains the bond NFT as input in their transaction, which would allow the attacker to perform those actions without the victim's consent, potentially leading to unauthorized access or manipulation of the victim's DeFi positions or rewards.

The impact of this attack depends on the specific protocol the attacker is targeting. For example, targeting a user's smart-wallet may have a lower impact than targeting a DEX pool or a lending pool, as the latter can affect multiple users and potentially cause more financial damage. The time sensitivity of the attack also matters, as blocking certain financial operations during critical moments (e.g. during a liquidation event) can have a higher impact than blocking them during less critical times.

So, the impact ranges from user inconvenience (e.g. blocking a user's smart-wallet) to significant financial damage (e.g. blocking a DeFi action during a critical moment).

2- UTxO-level Spam Attack

Since the `ThirdPartyAct` path allows including any UTxO from the smart-wallet as input, and allows adding new programmable token to the `value` of the output UTxO, an attacker can potentially add a large number of new tokens to the output UTxO, which can increase the execution cost of spending that UTxO in the future, potentially making it unspendable if there is not enough headroom in the execution budget.

The attacker can add spam tokens with a policy ID that is lexicographically less than the target policy ID, which would make the execution of the `ThirdPartyAct` path more expensive for the victim when they try to spend the UTxO containing the spam tokens, since the `split_at` function would have to iterate through all the tokens before the target policy ID, which can be a large number of tokens if the attacker adds many spam tokens.

This attack can add approximately 1.4M memory units and 400M CPU units to the execution cost of spending the UTxO containing the spam tokens. And if the `third_party_logic` execution budget for the victim's token depends on the value size of that UTxO, then the execution cost of the `ThirdPartyAct` path would be even higher.

The final impact of this attack is that if there is not enough headroom in the execution budget for the `ThirdPartyAct` path, the transaction would fail, and thus, the UTxO containing the spam tokens would be effectively locked for `ThirdPartyAct` path. And since the `TransferAct` path would also require satisfying the transfer logic of all programmable tokens in the same UTxO, the UTxO would be effectively locked for both `ThirdPartyAct` and `TransferAct` paths, i.e. completely unspendable.

The expected likelihood for this attack scenario is low to medium, depending on the final execution costs of the `ThirdPartyAct` path and the amount of headroom typically available. However, the impact of such an attack is high, as it will lead to a permanent lock of the affected UTxO.

Relevant code snippets:

`split_at` function:

```
1  pub fn split_at(
2      self: Value,
3      at: PolicyId,
4      return: fn(Assets, Tokens, Assets) -> result,
5  ) -> result {
6      do_split_at(from_value(self), at, [], return)
7  }
8
```

 Aiken

```

9  fn do_split_at(
10    self: Assets,
11    at: PolicyId,
12    before: Assets,
13    return: fn(Assets, Tokens, Assets) -> result,
14  ) -> result {
15    when self is {
16      [] -> return(before, dict.empty, [])
17      [Pair(k, _) as head, ..tail] ->
18        if less_than_bytearray(k, at) {
19          // Skip while keys are smaller (strictly) than searched key
20          do_split_at(tail, at, [head, ..before], return)
21        } else {
22          if k == at {
23            // Done searching, get the value and return the tail after
24            return(before, head.2nd, tail)
25          } else {
26            // The head and tail are all after the key; no need to continue searching.
27            return(before, dict.empty, self)
28          }
29        }
30    }
31  }

```

Here the `split_at` function recursively iterates through the consumed UTxO's value till it finds the target policy ID, returning the assets before the target policy, the tokens of the target policy, and the assets after the target policy. Thus, finding the target policy ID would be cheaper if the target policy ID is lexicographically less than the policy ID of the rest of the tokens in the value, as it would require iterating through fewer tokens. So, the attacker can add spam tokens with a policy ID that is lexicographically less than the target policy ID, which would make the execution of the `ThirdPartyAct` path more expensive for the victim when they try to spend the UTxO containing the spam tokens, since the `split_at` function would have to iterate through all the tokens before the target policy ID, which can be a large number of tokens if the attacker adds many spam tokens.

Test results for `split_at` function with 145 spam tokens:

```

1  └─ assets.test ───────────────────────────────────────────────────────────
2  | PASS [mem: 33.47 K, cpu: 99.24 M] split_at_test_A
3  | PASS [mem: 1.44 M, cpu: 501.04 M] split_at_test_V
4  └────────────────────────────────────────────────────────────────────────── 2 tests | 2 passed | 0 failed

```

Code used to generate the test results:

[illegible]

[illegible]

```

79     output_tokens_before,
80     output_tokens_at,
81     output_tokens_after,
82     <- assets.split_at(value_f, policy_id_end)
83
84     // Seized tokens must change, but only them.
85     expect and {
86         (input_tokens_before == output_tokens_before)?,
87         (input_tokens_after == output_tokens_after)?,
88         (input_tokens_at != output_tokens_at)?,
89     }
90 }

```

3- Combined Attack Vector

Combining the `ThirdPartyAct` vulnerability with the composability problem in `TransferAct` path (finding **F302**) creates a powerful attack vector:

By taking advantage of the `ThirdPartyAct` vulnerability (injecting malicious tokens into the victim's smart-wallet), and the `TransferAct` vulnerability (the composability problem), the attacker can create a malicious token with a malicious `transfer_logic_script`, and use the `ThirdPartyAct` to inject that malicious token into the victim's smart-wallet UTxO.

Attack Scenario:

1. An attacker creates a malicious token `A` with the following logic scripts:
 - A malicious `transfer_logic_script` that demands a ransom payment (e.g. 1000 ADA) to allow the transfer of the token.
 - A `third_party_transfer_logic_script` that allows only the attacker to use the `ThirdPartyAct` path for that token.
2. The attacker uses the `ThirdPartyAct` to inject the malicious token `A` into the victim's smart-wallet by including any UTxO from the victim's smart-wallet as input in their transaction.
 - The victim's smart-wallet is now infected with the malicious token `A`.
3. The victim's tokens are now held hostage by the malicious `transfer_logic_script` of token `A`, as the victim cannot use the `TransferAct` path to transfer their tokens without satisfying the ransom demand of token `A`.
 - The victim is forced to pay the ransom to the attacker in order to free his/her tokens using the `TransferAct` path.

Such an attack can affect any smart-wallet UTxO, including DEX liquidity pools, lending pools, collateral UTxOs of DeFi positions, NFT marketplaces, etc. The impact of this attack can lead to significant financial damage, especially if there are non-programmable tokens (including ADA) in the same UTxO, as there are no current mechanisms to free non-pro-

grammable tokens from a malicious `transfer_logic_script` (e.g. ADA-Programmable_token DEX pool, or any Non_programmable_token-Programmable_token DEX pool).

The main practical caveat for this attack is the min-ADA requirement. If adding the malicious token `A` makes the output require more min-ADA than what is currently in the UTxO, then the ledger would reject the transaction, and thus, the attack would fail. However, if the victim's UTxO has enough ADA headroom to accommodate the additional min-ADA requirement of the malicious token `A`, then the attack can succeed.

Another caveat is that the `ThirdPartyAct` can also be used to free tokens that are stuck because of a malicious `transfer_logic_script`, however, the `ThirdPartyAct` can only be used to free programmable tokens, and not non-programmable tokens (including ADA), so if the victim's tokens are non-programmable, then the `ThirdPartyAct` cannot be used to free them, and thus, the attack would be more effective. Moreover, executing the `ThirdPartyAct` path to free tokens may not be a practical solution for the victim, as the attacker can re-inject the same malicious token back into the victim's smart-wallet new UTxO, thus, the victim would be in a continuous loop of needing the admin to execute the `ThirdPartyAct` path to free the tokens, but the attacker keeps re-injecting the malicious token back into the victim's smart-wallet.

Mitigations

To mitigate the `ThirdPartyAct` vulnerability, the main constraint is to restrict the ability of the 3rd-party action to include any UTxO from the smart-wallet as input, and to restrict the ability of the 3rd-party action to add new tokens to the `value` of the output UTxOs with a unique policy ID that is not already present in the smart-wallet input UTxOs, as this would allow the 3rd-party action to inject new tokens into user's smart-wallet UTxOs.

The following implementation changes are recommended to mitigate the vulnerability:

1. In `check_seized_tokens` function:

In the `input.address.payment_credential == prog_logic_cred` branch, add the following check:

```
1 dict.is_empty(input_tokens_at) == False
```

 Aiken

This check ensures that the 3rd-party action can only modify the target policy tokens that are already present in the input UTxOs, and cannot add new tokens with a unique policy ID that is not present in the input UTxOs.

2. In `check_seized_tokens` function:

In the `input.address.payment_credential == prog_logic_cred` branch, the following check is redundant and can be removed:

In the `expect and` block:

```
1 (input_tokens_at != output_tokens_at)?,
```

 Aiken

This check does not add any additional security, since allowing the 3rd-party action to keep the same amount of the target policy tokens (i.e. `input_tokens_at == output_tokens_at`) would not cause any harm, as long as the 3rd-party action cannot add new tokens with a unique policy ID that is not present in the input UTxOs.

3. Off-chain mitigations:

- Users need to be well educated about the risks of consolidating smart-wallet UTxOs, as this could expose the whole UTxO (including all tokens in it) to a DOS attack via a malicious `third_party_transfer_logic_script`.
- Recommend transaction builders to split the user's tokens into multiple UTxOs, and avoid putting all tokens in a single UTxO.
- Wallet providers can implement a monitoring system to detect and alert users about any suspicious programmable token activity in their smart-wallets, provide a garbage collection mechanism to remove malicious/spam tokens from the user's smart-wallet, and provide a token quarantine mechanism to isolate suspicious tokens from the rest of the user's assets until they are verified to be safe.

These mitigations would significantly reduce the attack surface of the `ThirdPartyAct` vulnerability, and thus, mitigate the potential attack vectors that can arise from it. However, it is important to note that these mitigations may not completely eliminate all risks, and users should still exercise caution when interacting with programmable tokens and smart-wallets.

Resolution

Every positionally paired PLB input must already contain the acted-on policy, and its continuing output preserves the pair's full address, datum, reference script, and all non-target token policies. `ThirdPartyAct` can therefore no longer inject its target policy into a programmable UTxO that did not already contain it.

[H 401] Reference Script Preservation in ThirdPartyAct Path

	Level	Severity	Status
	4	High	Resolved

Category: Continuing Outputs

Affected File: `validators/programmable_logic/third_party.ak`

Description

In the `ThirdPartyAct` path, `check_seized_tokens` function pairs each input UTxO from the PLB address with an output UTxO, and checks that the address, datum, and value (except for the seized tokens) are preserved between the input and output. However, there is no check that the Reference Script of the output UTxO is the same as the Reference Script of the input UTxO.

Relevant code snippet from `third_party.ak`:

```

1  /// Analyze inputs and outputs affected by the 3rd-party action (grossly named
   *seized_tokens* here).
2  fn check_seized_tokens(
3    prog_logic_cred: Credential,
4    policy_id: PolicyId,
5    inputs: List<Input>,
6    outputs: List<Output>,
7    input_tokens: Tokens,
8    output_tokens: Tokens,
9  ) -> Bool {
10   when inputs is {
11     [] -> {...}
12     [Input { output: input, .. }, ..tail_inputs] ->
13       // Validate all inputs pointing at a programmable proof
14       if input.address.payment_credential == prog_logic_cred {
15         // Extract the paired output and ensures it preserves address and datum
16         let output = list.head(outputs)
17         expect output.address == input.address
18         expect output.datum == input.datum
19
20         // Extract seized tokens as the delta between outputs and inputs for the
21         // given policy, ensuring all else is equal.
22         let
23           input_tokens_before,
24           input_tokens_at,
25           input_tokens_after,

```

```

26     <- assets.split_at(input.value, policy_id)
27
28     let
29         output_tokens_before,
30         output_tokens_at,
31         output_tokens_after,
32     <- assets.split_at(output.value, policy_id)
33
34     // Seized tokens must change, but only them.
35     expect and {
36         (input_tokens_before == output_tokens_before)?,
37         (input_tokens_after == output_tokens_after)?,
38         (input_tokens_at != output_tokens_at)?,
39     }
40
41     check_seized_tokens(..)
42 } else {..}
43 }
44 }

```

Here the function checks that the address and datum are preserved between the input and output UTxO, but it does not check that the Reference Script is preserved. This means that an attacker can change the Reference Script of the output UTxO to a different script, which could potentially lead to unexpected behavior or security vulnerabilities.

Impact

If a UTxO holds a Reference Script that is expected to be preserved across the `ThirdPartyAct` transaction, an attacker could change the Reference Script in the output UTxO to a different script. This could lead to various issues, depending on the protocol's reliance on the Reference Script for certain functionalities.

Mitigation

Add the following check in the `check_seized_tokens` function to ensure that the Reference Script of the output UTxO is the same as the Reference Script of the input UTxO:

```
1 expect output.reference_script == input.reference_script
```

 Aiken

Add this check after the existing checks for address and datum preservation, to ensure that the Reference Script is also preserved between the input and output UTxO. This would prevent attackers from changing the Reference Script during a `ThirdPartyAct` transaction, thus maintaining the integrity of the UTxO's expected behavior.

Resolution

`ThirdPartyAct` now requires exact `reference_script` equality for every positionally paired programmable input and continuing output. Adding, removing, or replacing a reference script therefore fails validation.

[M 301] Enforcement of `transfer_logic_script` for Purely Minted Tokens

	Level	Severity	Status
	3	Medium	Resolved

Category: Design Contradiction / Inefficiency

Affected Files:

- `validators/programmable_logic_global.ak`
- `validators/programmable_logic/transfer.ak`

Description

The current design and implementation of CIP-113 contain a contradiction regarding when the `transfer_logic_script` is enforced. The intended design is that the `transfer_logic_script` should be enforced whenever a token is transferred (i.e., whenever the token is in the input from a smart-wallet) and should not be enforced when the token is purely minted (i.e., the token is only in the mint field and not in the input from a smart-wallet).

However, the current implementation contains a contradiction:

1. In the `validators/issuance_mint.ak`, the `transfer_logic_script` is not enforced when minting a token, which is correct and aligns with the intended design.
2. However, in the `validators/programmable_logic_global.ak` (specifically in the `validate_transfer` function), the `transfer_logic_script` is enforced if a token is purely minted, which is incorrect and contradicts the intended design.

Relevant code snippet from `validators/programmable_logic_global.ak`:

```
1  /// Validate a transfer action
2  pub fn validate_transfer(
3      self: Transaction,
4      prog_logic_cred: Credential,
5      get_registry_node: fn(Int) -> Data,
6      proofs: List<RegistryProof>,
7  ) -> Bool {
8      ...
9      let input_assets = collect_input_assets(self, has_withdrawal, prog_logic_cred)
10     ...
11 }
```

In this code, the `collect_input_assets` function collects the tokens from the transaction inputs that are sent from a smart-wallet (programmable + non-programmable), but it also collects the tokens that are being minted in the same transaction (programmable + non-programmable). This means that if a token is being purely minted (i.e., it is only in the mint

field and not in the input from a smart-wallet), it will be subject to the `transfer_logic_script` even though it is not being transferred, which is a contradiction to the intended design.

```
1 /// Get inputs from prog_logic_cred and verify ALL have proper authorization Aiken
2 /// This function enforces ownership by requiring that every input from prog_logic_cred
3 /// is authorized by its stake credential (signature or script invocation).
4 /// If ANY input lacks authorization, the transaction FAILS.
5 /// Sum values from a list of inputs
6 /// Check transfer logic for each currency symbol and filter to only programmable tokens
7 fn verify_proofs(..) -> List<RegistryProof> {..}
```

`verify_proofs` is mainly doing two things:

1. Enforcing `transfer_logic_script` execution for all programmable tokens in the transaction, including purely minted tokens.
2. Token balance check by comparing the tokens in the inputs and outputs (inputs must be in the outputs, i.e., the programmable tokens never leave the smart-wallet).

```
1 expect Aiken
2   tokens.contains(
3     dict.to_pairs(head_output_tokens),
4     dict.to_pairs(head_input_tokens),
5   )
```

Moreover, requiring a proof for purely minted tokens creates redundancy and inefficiency, since the minting policy is already performing checks to ensure that the token is being minted correctly.

Impact

1. Design Contradiction: The enforcement of the `transfer_logic_script` for purely minted tokens contradicts the intended design of CIP-113.
2. Unintended Consequences: This could lead to unexpected behavior or errors in transactions involving purely minted tokens, as the `transfer_logic_script` may impose constraints or requirements that are not relevant to the minting process. The `transfer_logic_script` may require certain conditions to be met (e.g., specific signatures, state checks) that may not be desirable or necessary for purely minted tokens, leading to failed transactions or unintended consequences for users minting new tokens.
3. Inefficiency: Requiring the `transfer_logic_script` to be enforced for purely minted tokens creates unnecessary overhead and inefficiency, as it requires additional script execution for tokens that are not being transferred. Moreover, it requires the user to provide proofs for purely minted tokens, which is redundant since the minting policy already performs necessary checks during the minting process.

Recommendations

In order to resolve this contradiction, the implementation should be updated to ensure that the `transfer_logic_script` is only enforced for tokens that are being transferred (i.e., tokens that are in the input from a smart-wallet) and not for tokens that are purely minted (i.e., tokens that are only in the mint field and not in the input from a smart-wallet). This could involve updating the `validate_transfer` function to differentiate between transferred tokens and purely minted tokens, ensuring that the `transfer_logic_script` is only applied to the former.

To achieve this, the following changes could be made:

1. Separate the 2 main functionalities in `verify_proofs` into two functions: one for enforcing `transfer_logic_script` execution and another for token balance checks. This would allow for more granular control over when the `transfer_logic_script` is enforced.
2. In the function responsible for enforcing `transfer_logic_script` execution, ensure that it only processes tokens that are in the inputs from a smart-wallet.
3. In the function responsible for token balance checks, ensure that it accounts for purely minted tokens appropriately, without subjecting them to the `transfer_logic_script`.

By implementing these changes, the design would align with the intended functionality of CIP-113, ensuring that the `transfer_logic_script` is enforced only for transferred tokens and not for purely minted tokens, thus resolving the contradiction in the current implementation. It would also improve the efficiency of the implementation by avoiding unnecessary enforcement of the `transfer_logic_script` and proof requirements for purely minted tokens.

Resolution

Purely minted policies are excluded from transfer proof verification, removing the original mint-only contradiction and redundant proof work.

Transfer validation now builds its proof domain from policies present in programmable inputs and applies mint or burn deltas only to those policies. Regression tests confirm that pure mints require neither a `TransferAct` proof nor `transfer_logic_script` invocation and reject unnecessary surplus proofs.

[M 302] Composability Problem in TransferAct Path

	Level	Severity	Status
	3	Medium	Mitigated

Category: Composability

Affected Files:

- `validators/programmable_logic_global.ak`
- `validators/programmable_logic/transfer.ak`

Description

The CIP-113 design allows every programmable token to define its own token-specific `transfer_logic_script`. When a `TransferAct` transaction spends UTxOs from `programmableLogicBase`, the global validator must prove the registration status of every relevant token policy and, for each registered programmable token, require the corresponding `transfer_logic_script` to be executed.

This means that a transaction spending a PLB UTxO containing multiple programmable tokens must satisfy the transfer logic of all those tokens simultaneously.

This creates a composability challenge: The more programmable tokens exist, the tighter the constraints on transaction shapes become, as each token's transfer logic may impose unique requirements on outputs, inputs, signatories, validity ranges, minting, datum shapes, or other assets. Even if individual tokens have reasonable transfer logic, their combination can lead to conflicting requirements that make it hard or even impossible to satisfy.

No standard implementer can account for all current and/or future transfer logic scripts. That means that the composability of protocols implementing the standard depends on the intersection of all these scripts' accepted transaction shapes.

Impact

In cases where a PLB UTxO contains multiple programmable tokens with conflicting transfer logic, the UTxO may become effectively locked for `TransferAct` path, as no valid transaction can satisfy all the required transfer logic scripts simultaneously.

Other cases include, DeFi actions that require multiple tokens (e.g. DEX swaps, lending and borrowing actions, collateral management, etc.) may become difficult or impossible to execute if the involved tokens have conflicting transfer logic.

At the level of the protocols implementing the standard, this adds complexity and fragility, as they need to account for the possibility of conflicting transfer logic scripts when designing their interactions with programmable tokens. This increases the risk of bugs, unexpected behavior, potential security vulnerabilities, and auditing costs.

An attacker can take advantage of this composability problem to create tokens with malicious transfer logic scripts that impose unreasonable requirements on transactions, with the intention of causing denial of service or extortion.

Attack Scenario

- An attacker creates a malicious token `A` with the following transfer logic:
 - 1000 ADA must be present in the outputs for the transfer to be valid.
 - Input UTxOs can be merged with other tokens, but cannot be unmerged without satisfying the above condition.
- The attacker sends token `A` to the victims' smart-wallet (At this stage token `A` is in a separate UTxO).
- The victim merges token `A` with their own tokens in the same UTxO.
- Now the victim's UTxO is infected with token `A`, and thus, spending that UTxO requires satisfying the malicious `transfer_logic_script` of token `A`.
- The victim is forced to pay 1000 ADA to the attacker in order to free his/her tokens using `TransferAct`.

Conflict Probability

Factors that increase the likelihood of conflicts include:

- The number of programmable tokens in the ecosystem, as more tokens increase the chances of incompatible transfer logic.
- The complexity of transfer logic scripts, as more complex logic may have more specific requirements that can conflict with others.
- The use of multi-token UTxOs, which are common in wallets, DEX pools, and DeFi positions, increasing the chances of encountering multiple tokens with conflicting logic.
- Protocols that require interactions with multiple tokens simultaneously, such as DEX swaps, NFT marketplaces, or lending and borrowing platforms, further increase the likelihood of conflicts.

While the probability of conflict between any two random tokens may be low, the combinatorial explosion of possible token combinations in real-world scenarios makes it likely that conflicts will occur. This is especially true in an open ecosystem where anyone can create programmable tokens with custom transfer logic. Moreover, the way DeFi protocols often require interactions with multiple tokens simultaneously (e.g. DEX swaps, multi-asset DeFi actions) further increases the likelihood of encountering such conflicts.

Recommendation

Although `ThirdPartyAct` can be used as a workaround to transfer tokens with conflicting transfer logic, it is not a practical alternative for composability.

Moreover, `ThirdPartyAct` can only be used to transfer programmable tokens out of the PLB UTxO, but it cannot be used to transfer ADA or other non-programmable tokens.

This means that if a PLB UTxO contains both programmable tokens and ADA, or other non-programmable tokens, the composability problem still exists for the ADA and non-programmable tokens, as they cannot be transferred using `ThirdPartyAct` and would still require satisfying the transfer logic of all programmable tokens in the same UTxO.

Off-chain mitigations are also possible by trying to separate programmable tokens into different UTxOs. However, off-chain solutions are preventive rather than reactive, and makes the standard security reliant on off-chain behavior rather than on-chain guarantees, which is not ideal.

Therefore, the design needs to allow actions to transfer single programmable tokens without the help of `ThirdPartyAct` or relying on off-chain solutions.

To mitigate the composability problem, the design needs to allow the transfer of individual programmable tokens without requiring `transfer_logic_script` execution for all other programmable tokens in the transaction.

This can be achieved by adding a new redeemer action that allows a transaction to specify that only one token is being transferred, while the other tokens in the same PLB UTxO are effectively “passed through” without requiring their transfer logic to be executed. This would allow users to split apart conflicting tokens and enable more flexible transaction shapes.

For example, the redeemer could specify the one token that is being transferred, and the validator would check that the other tokens in the same PLB UTxO are returned to the same PLB address with the same datum, value (minus the transferred token), and the same `ReferenceScript`. This way, the transaction can still be valid even if there are multiple conflicting programmable tokens in the same PLB UTxO.

Although this mitigation solves the composability problem for conflicting tokens in a single UTxO, it does not entirely solve the problem for transactions that need to transfer multiple programmable tokens at once (e.g. DEX actions, or DeFi actions that need to involve multiple tokens simultaneously). However, it prevents the most critical case of UTxOs becoming completely unspendable by `TransferAct` due to conflicting transfer logic.

Resolution

`UnfrackingAct` lets an authorized holder fully separate one registered policy from same-owner mixed PLB UTxOs without invoking co-resident transfer scripts, provided that policy’s issuer-configured unfracking hook authorizes the action. This mitigates collateral trapping for opted-in policies, while non-acted token assets remain in the continuing UTxO; ADA may be rebalanced during a successful split. `TransferAct` still requires the conjunction of every participating policy’s logic, so conflicting scripts can continue to block atomic multi-token transactions.

[M 303] Unconstrained Asset Addition in Smart-Wallets

	Level	Severity	Status
	3	Medium	Mitigated

Category: Denial of Service / Resource Exhaustion / UTxO Bloat / Asset Spam

Affected Files:

- `validators/programmable_logic_global.ak`
- `validators/programmable_logic/third_party.ak`

Description

In the `ThirdPartyAct` path of the `programmable_logic_global` validator, the `validate_3rd_party` function checks that the target policy in the value of the continuing outputs is changed, but does not impose any constraints on the number of assets that can be added.

If a large number of assets are added to the continuing outputs, it may lead to an increase in execution costs when consuming that UTxO in the future. However, this increase is not problematic for the current implementations of the `ThirdPartyAct` path, as the `validate_3rd_party` function does not traverse the assets with the same policy.

However, this issue could become problematic if the execution cost of the `third_party_transfer_logic_script` for the target policy depends on the number of assets with the same policy. Or the `transfer_logic_script` execution cost depends on the number of assets in the value of the continuing outputs.

This problematic state can be reached not only by the `ThirdPartyAct` path, but also by any other path that allows adding assets to the continuing outputs without constraints. For example, during minting, transfer, or both of them, a large number of assets can be added to the continuing outputs, which can lead to the same issue.

Impact

A UTxO with a large number of assets can lead to increased execution costs when consuming that UTxO in the future, which can be problematic for users and applications interacting with the system.

The value in a single UTxO can have approximately 1230 assets (a value with 1 ADA, 1 token with policy A, and 1230 tokens with policy B).

Benchmarks

1	<code>assets.test</code>	txt
2	PASS [mem: 33.47 K, cpu: 12.82 M] <code>attacker_add_1230_tokens_same_policy</code>	
3	PASS [mem: 39.23 K, cpu: 349.69 M] <code>victim_remove_target_token_different_policy</code>	
4	2 tests 2 passed 0 failed	

In this benchmark, attacker adds 1230 tokens (Policy B), then victim removes the target token (Policy A). The execution cost of the victim's transaction is significantly higher than

the attacker's transaction, which demonstrates the potential increase in execution costs when consuming a UTxO with a large number of assets.

These numbers only represent the execution cost of the `split_at` function, which is only a part of the `ThirdPartyAct` path. The total execution cost of the `ThirdPartyAct` path can be significantly higher, especially if the `third_party_transfer_logic_script` execution cost increases with such a state.

Recommendation

1. To mitigate this issue, we recommend introducing an `Unfracking` action in the PLGlobal redeemer. This action would allow users to redistribute the tokens over multiple UTxOs without fundamentally changing the distribution of value and ownership. This action should be separate from the `ThirdPartyAct` path, as it should not be used for compliance operations, but rather as a tool for users to manage their UTxOs and avoid fragmentation issues.
2. Alternatively, if the `Unfracking` action is not introduced, we recommend imposing constraints on the number of assets that can be added to the continuing outputs in any path that allows adding assets. This could involve setting a maximum limit on the number of assets or implementing a cost mechanism that increases with the number of assets added. Such paths include the `ThirdPartyAct` path, minting paths, and transfer paths. Note, setting the maximum limit to a value that is too low may limit the usability of the smart-wallets, while setting it to a value that is too high may not effectively mitigate the issue. Therefore, careful consideration should be given to determining an appropriate limit.

Resolution

`ThirdPartyAct` now prevents foreign-policy additions to paired continuing outputs and requires each paired input to already contain the acted-on policy, but it still places no asset-name limit on that policy. `UnfrackingAct` can redistribute the policy's asset set only when its issuer-configured hook authorizes the action; because that hook may be unset or deny recovery, the issue is mitigated rather than resolved.

[L 201] Registry Init Does Not Bind Origin Node to `registry_spend`

	Level	Severity	Status
	2	Low	Resolved

Category: Bootstrap Trust / Registry Custody

Affected Files:

- `validators/registry_mint.ak`
- `lib/linked_list.ak`

Description

On `RegistryInit`, `registry_mint` calls `validate_directory_init`, which checks the one-shot spend, that no registry inputs are spent, a single node output with correct value/datum/NFT shape, origin list semantics, and the origin registry NFT in the mint. It does not require that output to be locked at the `registry_spend` address. A mistaken bootstrap tx can therefore place the origin registry node at any address while still passing validation.

Relevant code snippet

In particular, `collect_node_ios` anchors node outputs to the address of the first node input, but only if such an input already exists:

```
1  when node_inputs is {
2    [first, ..] -> {
3      let expected_address = first.output.address
4      expect
5        list.all(
6          node_outputs,
7          fn(output) { output.address == expected_address }, // check that node outputs
                        are at the same address as the first node input
8        )
9      (node_inputs, node_outputs)
10   }
11   [] -> ([], node_outputs)
12 }
```

This means `RegistryInsert` preserves the address of the covering node input, but `RegistryInit` has no such anchor because there are no existing node inputs yet.

Exploit scenarios

- During bootstrapping (`RegistryInit`), the operator sends the origin registry node to an address they control instead of the canonical `registry_spend` address.
- Later, they can perform `RegistryInsert`. In `RegistryInsert`, the validator only enforces that node outputs stay at the same address as the first node input, so the transaction

validation passes, with the newly inserted registry nodes also remaining at the attacker-controlled address.

- They can then send an origin node UTXO back to the `registry_spend` address with the correct NFT and output format.

At that point, some authentic registry NFTs have existed outside the canonical registry lock. If the protocol continues operating and more nodes are registered or more programmable tokens are minted, this opens two exploit scenarios.

Scenario 1: Authority hijack via poisoned registry node

- The attacker uses the registry NFTs they control to create or reintroduce a registry node with a malicious datum.
- In that forged datum, they can replace `third_party_transfer_logic_script` with a script they control.
- PLG will trust that registry node as the authority for that policy.
- The attacker can then invoke `ThirdPartyAct` and seize users' programmable-token UTXOs for that policy, subject to the remaining accounting checks.

Scenario 2: Programmable-enforcement bypass via forged non-membership interval

- The attacker forges registry node keys / gaps so that a genuinely registered policy can be treated as if it were not included in the registry.
- This allows them to construct a fake `TokenDoesNotExist` -style non-membership proof.
- PLG then treats that policy as non-programmable and skips the normal programmable-token containment checks for it.
- As a result, the token can be moved outside programmable custody in otherwise authorized transfer flows.

Impact

This is not a permissionless outsider issue. Exploitation requires malicious or compromised control of the bootstrap flow, or collusion with the party that receives the misdirected origin node.

However, if that happens, the impact is high:

- The registry custody invariant is broken at genesis
- Authentic registry NFTs can remain under attacker-controlled custody outside the canonical `registry_spend` lock
- Poisoned registry nodes can later be reintroduced and trusted by downstream validators, and
- The attacker can abuse that trust either to hijack third-party authority for a policy or to make a registered policy appear absent from the registry.

This is therefore a bootstrap / insider trust failure with protocol-wide consequences: once the canonical registry lineage is broken, downstream validators may continue trusting poisoned registry state.

Recommendation

Enforce the registry custody address on-chain during `RegistryInit`.

Specifically:

- Store the `registry_spend` address or payment credential in a protocol param UTxO datum
- Include the protocol-params currency symbol as a parameter of `registry_mint`,
- Require a protocol-params reference input in `RegistryInit`,
- Read the expected `registry_spend` address or payment credential from that protocol-params datum, and
- Assert that the origin registry node output is locked at that expected address.

This ensures the registry lineage starts under the canonical registry lock and prevents attacker-controlled or non-canonical custody from being established at bootstrap.

Resolution

`registry_mint` now receives the trusted `registry_spend` credential as a validator parameter and requires the sole origin-node output's full address to match it. Insert and update paths preserve that anchored address, closing the bootstrap-custody defect.

[L 202] Registration and Transfer Coupling Issue

	Level	Severity	Status
	2	Low	Resolved

Category: Design Limitation / Token Provability

Affected File: `validators/programmable_logic/transfer.ak`

Description

In the current design, to register a token, the user needs to mint it in the same transaction. This means that this token will appear in the mint field of the transaction. For example, if the user wants to register token A, the user needs to mint token A in the same transaction. This means that `self.mint` will include token A in its list of tokens.

In the PLGlobal validator, in the `TransferAct` branch, function `validate_transfer` needs to prove inclusion or exclusion for every token in the PLB inputs and mint. This means that if a token is minted in the same transaction, it will be included in the `input_assets` and will be checked against the registry using the `choose_registered_token_with` function.

In the `choose_registered_token_with` function, there are 2 proof options for each token:

- `TokenExists`: The proof includes a node index that points to a registry node. The validator checks that the node's key matches the token's policy and that the transfer logic script is invoked. If both checks pass, the token is considered a registered programmable token.
- `TokenDoesNotExist`: The proof includes a node index that points to a registry node. The validator checks that the node's key and next key cover the token's policy (i.e., `node.key < policy < node.next`). If this check passes, the token is considered not a programmable token.

Problem

Since registering a token requires minting it in the same transaction, this means that the token being registered will be included in the `input_assets` and will be checked against the registry.

However, the `choose_registered_token_with` function will not be able to prove neither the inclusion nor the exclusion of the token being registered, as for proving inclusion, the registry node is not yet added to the registry, so it cannot be included as a `ref_input`, and for proving exclusion, the previous node in the registry is consumed to update its `next` field to point to the new node, so it cannot be included as a `ref_input` as well. This means that the proof for the token being registered will always fail, and thus the transaction will always fail.

So, we cannot prove neither the inclusion nor the exclusion of the token being registered in the same transaction.

Relevant Code Snippet

```

1  // Collect all programmable tokens across inputs and minted/burned values.
2  let input_assets = collect_input_assets(self, has_withdrawal, prog_logic_cred)
3
4  // Check transfer logic for each programmable token type and filter to only
   programmable tokens.
5  [] == verify_proofs(
6    proofs,
7    choose_registered_token_with(get_registry_node, has_withdrawal, _, _, _, _),
8    input_assets,
9    output_assets,
10 )

```

In this code snippet, `input_assets` includes all tokens from the smart-wallet inputs and all minted/burned tokens. This means that if a token is minted in the same transaction, it will be included in the `input_assets` and will be checked against the registry using the `choose_registered_token_with` function:

```

1  /// Validate a single currency symbol against its proof
2  /// Returns True if the token IS a programmable token (TokenExists with valid proof)
3  /// Returns False if the token is NOT a programmable token (TokenDoesNotExist with
   valid proof)
4  /// FAILS the transaction if any proof is invalid
5  fn choose_registered_token_with(
6    get_registry_node: fn(Int) -> Data,
7    has_withdrawal: fn(Credential) -> Bool,
8    policy: PolicyId,
9    proof: RegistryProof,
10   when_registered: fn() -> result,
11   when_not_registered: fn() -> result,
12 ) -> result {
13   when proof is {
14     TokenExists { node_idx } -> {
15       let
16         key,
17         transfer_logic_script,
18         <- with_key_and_transfer_logic(get_registry_node(node_idx))
19
20       // Validate the node's key matches the currency symbol
21       expect key == policy

```

```
22
23     // Validate the transfer logic script is invoked
24     expect has_withdrawal(transfer_logic_script)
25
26     when_registered()
27 }
28 TokenDoesNotExist { node_idx } -> {
29     let key, next <- with_key_and_next_key(get_registry_node(node_idx))
30
31     // Validate the node covers the currency symbol (node.key < policy < node.next)
32     expect less_than_bytearray(key, policy)
33     expect less_than_bytearray(policy, next)
34
35     // This is NOT a programmable token (proof is valid)
36     when_not_registered()
37 }
38 }
39 }
```

Impact

- Registering and `TransferAct` transfer cannot happen in the same transaction. So Registering a token will require a separate transaction that do not include any `TransferAct` transfer.
- While this issue does not create a security vulnerability, it may limit some features of the current design, such as the ability to register and transfer a token in the same transaction.

Remediation options

Option 1: Separation of concerns

- Apply the recommendation of finding **F103** (separation of concerns), which will separate the registration and issuance of tokens into two separate actions. This will improve design efficiency, security, and lower design complexity, but it will limit some features of the current design, such as the ability to register and transfer a token in the same transaction.
- Check finding **F103** for more details.

Option 2: Expand the search space for the proof

- Expand the search space for the proof to include output UTXOs as well, since the updated nodes can be found in the outputs. This will increase execution cost, but it will not limit the functionality of the current design.
- This way, the `choose_registered_token_with` function can find the updated node in the outputs and prove the inclusion of the token being registered in the same transaction.

Option 3: skip the proof for the token being registered in the same transaction

- Delegate the proof of inclusion for the token being registered to the issuance logic.
- Check whether the `redeemer.minting_registry_proof` is of type `OutputIndex`, and if so, skip the proof for that token in the `choose_registered_token_with` function, as the proof will be handled in the issuance logic.
- This way, we can allow the registration and transfer of a token in the same transaction without needing to expand the search space for the proof, but it may add some complexity to the design as we need to scan `self.redeemers`.

Resolution

Transfer proofs are now derived from policies present in programmable inputs, so a newly registered policy appearing only in mint is excluded from the impossible reference-input proof. Registration works alone or with a first mint, while issuance independently authenticates the policy and enforces programmable custody.

[I 101] Ambiguity in the Scope of Programmable Control

Level	Severity	Status
1	Informational	Resolved

Category: Design Ambiguity / Specification Scope / Standards Interoperability

Affected Files:

- `validators/issuance_mint.ak`
- `validators/registry_mint.ak`
- `validators/programmable_logic/transfer.ak`
- `lib/registry_node.ak`

Description

The current CIP-113 design and implementation create ambiguity around the scope of programmable control, particularly regarding whether metadata management and royalty management are intended to be part of the standard's control surface or are out-of-scope features that should be handled by separate standards layered on top.

The CIP-113 documentation focuses on programmable control over token custody, ownership movement, and issuance flows, with a clear emphasis on transfer logic, third-party transfer logic, issuance logic, and optional global state. However, the current implementation appears to extend this control to include metadata management and royalty management as well, without explicit specification-level guidance on whether these features are intended to be part of the core CIP-113 framework or are adjacent standards that should be implemented separately.

CIP-68 and CIP-102 are examples of adjacent standards that introduce companion assets (e.g. reference NFTs for metadata, royalty NFTs for royalties) that may be intended to operate under the same policy as the user token but with separate control rules. The current CIP-113 framework does not explicitly define how such companion assets should be treated, leading to ambiguity about whether they are fully programmable assets under CIP-113, companion assets with separate control rules, or out-of-scope assets handled by another standard layered on top.

This design implementation creates the following conflict:

1. CIP-113's token control logic is applied at the policy level, meaning that all tokens under a registered policy are subject to the same programmable control rules, without differentiation between user tokens and companion assets.
2. Whereas CIP-68 and CIP-102's implementations are at the token level, allowing for separate control rules for different token roles (e.g. user tokens vs reference NFTs vs royalty NFTs) under the same policy.

Relevant Code Snippets

```
1 pub type RegistryNode {  
2     key: ByteArray,  
3     next: ByteArray,  
4     transfer_logic_script: Credential,  
5     third_party_transfer_logic_script: Credential,  
6     global_state_cs: ByteArray,  
7 }
```

Aiken

This design shows that the registry node is structured around policy-level registration and control, without explicit fields for differentiating between user tokens and companion assets.

Impact

Documentation Ambiguity: The lack of explicit specification-level guidance on the scope of programmable control creates ambiguity for developers and implementers, leading to inconsistent interpretations of the standard and potential misalignments between the intended design and actual implementations. **Standards Interoperability:** The extended control scope in the current implementation may lead to interoperability issues with adjacent standards like CIP-68 and CIP-102, which assume separate control rules for companion assets. This can lead to an overpowering of the CIP-113 framework and unintended consequences for developers building around these adjacent standards.

Recommendations

1. **Document Clarification:** The CIP-113 specification should be updated to explicitly clarify the intended scope of programmable control, clearly defining whether metadata management and royalty management are part of the core CIP-113 framework or are out-of-scope features that should be handled by separate standards layered on top.
2. **Design Choice:** Limit the scope of programmable control in CIP-113 to ownership change and transfer rules, and explicitly exclude metadata management and royalty management from the standard's control surface. This would align CIP-113 with common practices in other ecosystems (e.g. Ethereum) and allow for clearer boundaries between different types of assets and their respective control rules.
3. **Extended Control Implementation:** Extended control over metadata management and royalty management can still be achieved through separate standards as an extension standards layered on top of CIP-113 (a.k.a. Substandards).
4. **Design Alignment:** If metadata management and royalty management are intended to be out-of-scope for CIP-113, the design should be updated to reflect this by allowing for separate control rules for companion assets (e.g. reference NFTs, royalty NFTs) under the same policy, without subjecting them to the same programmable control rules as user tokens. This could involve updating the registry node structure to include explicit fields

for differentiating between user tokens and companion assets, allowing for separate control rules to be defined for each type of asset.

- Implementation Notes: To achieve this design change, the registry node structure could be updated to include an optional inclusion list or exclusion list that specifies which tokens under the policy are subject to programmable control and which are not. This could be implemented by adding an optional field to the registry node structure that references an external inclusion/exclusion list (e.g. a separate on-chain data structure or a Merkle tree), allowing for flexible and scalable management of control rules for different types of assets under the same policy.

References

- [CIP-68](#)
- [CIP-102](#)

Resolution

The new `documentation/03-CONTROL-SCOPE-AND-ADMIN-AUTHORITY.md` and the companion-asset guidance added to `documentation/09-DEVELOPING-SUBSTANDARDS.md` now clarify that CIP-68 metadata management and CIP-102 royalty management are outside CIP-113's programmable-control scope and are the responsibility of companion-aware substandards. They also explain that those substandards can apply the required token-level rules to companion assets under the same policy, resolving the ambiguity over which layer governs their metadata and royalty semantics.

Resolved in commit hash `17ef775ed0bc6ee9235f84c59294b1a3f3e72bf5`

[I 102] Linked List Registry Limitation

	Level	Severity	Status
	1	Informational	Acknowledged

Category: Liveness / UX Limitation / Denial of Service

Affected File: `lib/registry_node.ak`

Description

The CIP-113 design relies on the fact that the smart-wallet can prove the programmability of a token by referencing a node in the registry (Proof of inclusion or exclusion). This check is needed to consume any UTxO from the smart-wallet.

To register token N, the previous node (N-1) in the registry should be consumed to update its `next` field to point to the new node (N).

This design will break the transfer logic of token (N-1) if the node (N-1) is being consumed in the same transaction (or by a different transaction), as the proof of inclusion or exclusion of node (N-1) will fail.

Note: This issue is a known limitation of the on-chain linked list design. Nevertheless, the client and users need to be aware of this issue as it can lead to consequences that range from a bad user experience to a potential attack vector (depending on the use case and design of the protocol that is using CIP-113).

Relevant Code Snippet

```

1  /// Registry node in the linked list of registered programmable token policies 
2  /// This represents an entry in the CIP-0143 registry
3  pub type RegistryNode {
4      /// The key (currency symbol) of the programmable token policy
5      key: ByteArray,
6      /// The next key in lexicographical order (for linked list traversal)
7      next: ByteArray,
8      /// The transfer logic script credential that validates token transfers
9      transfer_logic_script: Credential,
10     /// The third party transfer logic script credential that handles admin actions
11     /// (seizure, etc.)
12     third_party_transfer_logic_script: Credential,
13     /// Optional global state NFT currency symbol for this policy
14     global_state_cs: ByteArray,
15 }

```

Impact

- User experience: Users will need to wait for the transaction that consumes node (N-1) to be confirmed before they can re-attempt the transaction that depends on node (N-1). This can lead to a bad user experience as users may not understand why their transaction is failing and may need to wait for an extended period of time before they can re-attempt the transaction.
- Attack vector: Depending on the use case and design of the protocol that is using CIP-113, this issue can be exploited by an attacker to create a denial of service attack. The severity of this attack vector depends on how much the system is sensitive to timing and delay, and how critical the affected operations are. For example, if the protocol is designed in a way that requires users to perform time-sensitive operations (e.g. trading, auctions, etc.), an attacker can exploit this issue to create a denial of service attack by consuming node (N-1) in a transaction and preventing users from performing their time-sensitive operations.

Remediation Options

Option 1: Backup registry with different data structure

- Implement a second registry with a different data structure (e.g. an array-like structure or a Merkle tree structure) that does not require consuming the previous node to add a new node.
- When adding a new node, both registries can be updated in the same transaction. While updating the linked list requires consuming the previous node, updating the second registry does not require consuming any node.
- The first registry (linked list) can be used for proof of inclusion or exclusion, while the second registry can be used as a backup registry to check the programmability of the token.
 - An array-like structure can prove only inclusion, while a Merkle tree structure can prove both inclusion and exclusion.
 - Note on implementing a Merkle tree structure: If we have a single root hash on-chain, we will face a contention problem. To mitigate this, we can have the root hash in an array-like structure on-chain, allowing us to update the root hash by adding a new hash to the array without needing to consume any node.
- This solution may add redundancy, complexity, and cost to the system, but it will solve the problem of the attack vector and the bad user experience.

Option 2: Separation of concerns

- Apply the recommendation of finding **F103** (separation of concerns), which will separate the registration and issuance of tokens into 2 separate actions.

- While this will solve both problems (inclusion and exclusion) during the same transaction, it is still possible that node (N-1) is being consumed by a different transaction, which can lead to the same issue.

Severity Justification

The severity of this issue is classified as “Informational” because it is a known limitation of the on-chain linked list design. However, it is important for clients and users to be aware of this issue as it can lead to consequences that range from a bad user experience to a potential attack vector (depending on the use case and design of the protocol that is using CIP-113). The impact of this issue can vary greatly depending on the specific use case and design of the protocol, and it may not always be exploitable as an attack vector. Therefore, it is classified as “Informational” rather than a higher severity level.

References

- [Merkle Patricia Forestry](#)

Resolution

The predecessor-node contention and resulting retry window remain, with no alternate authenticated proof source. The limitation is explicitly documented, off-chain re-resolution and transaction rebuilding are prescribed, and the client deliberately accepted the sequential workflow rather than changing the registry design.

[I 103] Separation of Concerns

Level	Severity	Status
 1	Informational	Resolved

Category: Design Simplification

Affected Files:

- `validators/issuance_mint.ak`
- `validators/registry_mint.ak`

Description

In the current implementation of CIP-113, the registration of a programmable token policy and the issuance (minting) of the programmable tokens are coupled together in the same transaction. This means that to register a token, the user needs to mint it in the same transaction. This design choice has some trade-offs and may not be ideal for all use cases.

Benefits of the current design:

- Minting proves control: It provides a proof of the instance of the minting policy during registration, as the minting logic credential is invoked during registration. This can help to prevent spammy registrations that do not have real tokens (e.g. a malicious actor can register a token with a malicious transfer logic script and third-party transfer logic script without actually minting any tokens, which can lead to potential security risks).
- User experience: It can provide a better user experience, as it allows for registration and issuance in the same transaction, which can be more user-friendly for some use cases. However, this may not be a strong benefit as in most cases, this action is a one-time action for the whole policy of the token, and is not a user-facing action, so it may not be a big deal to have it in separate transactions.

Drawbacks of the current design:

- Protocol limitations:
 - In some use cases, it may not be desirable to mint during registration.
 - For example, if a protocol wants to register a token that represents a real-world asset, it may not be able to mint that token during registration as it may not have the necessary information or data to mint the token at that time. In such cases, it may be better to have a clear separation of concerns between registration and issuance, allowing the protocol to register the token first and then mint it later when it has the necessary information or data.
 - Although, it is possible to create a separate minting path at the instance level `minting_logic_cred`, this would add complexity to the users implementing this standard, and may create some design limitations.
- Increased protocol complexity: Coupling two separate actions together can increase the complexity of the protocol, as it requires handling both registration and issuance logic in the same transaction, which can lead to increased execution cost and potential edge cases that need to be handled.

- **Lack of flexibility:** Coupling registration and issuance together can limit the flexibility of the protocol, as it may not allow for certain use cases that require a clear separation of concerns between registration and issuance.

Overall, while the current design of coupling registration and issuance together has some benefits, it also has some drawbacks that may not be ideal for all use cases. It is important to consider the trade-offs and potential limitations of this design choice when implementing CIP-113 in different protocols.

Questions to Consider

1. Is there an attack vector if we allow registration without minting?
2. What security risks are we trying to mitigate by requiring minting during registration?

Answering these questions can help guide the design decisions and ensure that the protocol is secure and flexible enough to accommodate different use cases.

Alternative Options to Consider

1. **Minting-independent registration:** Free the registration process from the minting process, allowing for registration without minting.
 - Keep minting during registration optional, but do not enforce it on-chain. This way, we can allow for registration without minting, while still providing the option for minting during registration for those use cases that may benefit from it.
 - This may allow for spammy registrations that do not have real tokens, and including malicious `transfer_logic_script` and `third_party_transfer_logic_script`, but it may provide more flexibility for different use cases.
2. **Proof of instance without minting:** Instead of requiring actual minting during registration, we can prove the instance of the issuance minting policy by requiring the invocation of the `minting_logic_cred` during registration without actually minting any tokens. This way, we can provide a proof of the instance of the minting policy during registration, without actually minting the issuable tokens during registration. This may be a good option as it allows for separation of concerns while still providing a proof of the instance of the minting policy during registration.
3. **Separate registration and issuance paths in the `issuance_mint` validator:** Separate the registration and issuance paths in the `issuance_mint` validator:
 1. The registration path: Require a one-time minting of a special token (e.g. “registration certificate NFT” or “proof of registry NFT”) that can be used to verify the legitimacy of the registry. This token can only be minted once and is used to prove that the registration is legitimate. Here we can also have running of the `minting_logic_cred` optional. Not requiring the `minting_logic_cred` to be executed during registration will help users implementing this standard to avoid the complexity of having a separate minting path at the instance level `minting_logic_cred`, so the `minting_logic_cred` can be reserved for the issuance path only.

2. The issuance path: Handle the actual minting of tokens according to the `minting_logic_cred`. This path is separate from the registration path and is responsible for enforcing the rules and logic for token issuance. We require that the token being minted is already registered in the registry.

Resolution

Registration is now independent of programmable-token issuance. Registry insertion cryptographically binds the minting-logic credential to the policy ID and separately requires that credential's withdrawal, preserving proof-of-instance authorization without a first mint. Registration-only, optional first-mint, and later-issuance flows remain supported, with issuance custody enforced by `issuance_mint`.

[I 104] Inefficient Membership Check in `issuance_mint` Validator

	Level	Severity	Status
	1	Informational	Resolved

Category: Optimization

Affected File: `validators/issuance_mint.ak`

Description

In the `issuance_mint` validator, there is a check for whether the `minting_logic_cred` is invoked during registration. This check is currently implemented by traversing the list of withdrawals in the transaction and creating an intermediate list of credentials, which is then checked for membership against the `minting_logic_cred`. This approach is inefficient as it traverses the list of withdrawals twice and allocates an intermediate list, which can lead to increased execution cost.

Relevant Code Snippet

```
1 // Collect all invoked scripts (withdraw-zero pattern)
2 let invoked_scripts =
3   list.map(
4     self.withdrawals,
5     fn(wdrl) {
6       let Pair(cred, _amount) = wdrl
7       cred
8     },
9   )
```

Aiken

Later in the code, there is a check for membership:

```
1 // substandard minting invoked?
2 list.has(invoked_scripts, minting_logic_cred)?,
```

Aiken

Recommendation

Instead of creating an intermediate list of credentials, we can directly check for membership in the original list of withdrawals without creating a new one. This can be done by iterating through the list of withdrawals and checking if any of the credentials match the `minting_logic_cred` directly, which would be more efficient and reduce execution cost.

Implementation of the recommended change could look like this:

```
1 let minting_logic_invoked =
2   list.any(
3     self.withdrawals,
4     fn(wdrl) {
```

Aiken

```
5     let Pair(cred, _amount) = wdrl
6     cred == minting_logic_cred
7 },
8 )
```

Then, instead of checking `list.has(invoked_scripts, minting_logic_cred)`, we can directly check the `minting_logic_invoked` variable:

```
1 // substandard minting invoked?
2 minting_logic_invoked,
```

 Aiken

This change eliminates the need for an intermediate list and reduces the number of traversals, improving the efficiency of the validator.

Resolution

The intermediate withdrawal-credential list and second membership traversal were removed. `issuance_mint` now uses a direct `list.any` predicate, and no reachable legacy map-then-membership path remains.

[I 105] Indirect Delegation Signal in PLB-Input Scan

	Level	Severity	Status
	1	Informational	Resolved

Category: Optimization

Affected File: `validators/issuance_mint.ak`

Description

In the `issuance_mint` validator, there is a condition that checks for the presence of a PLB input in the transaction's inputs to determine if mint custody is being delegated to PLGlobal. This check is implemented by scanning through the transaction inputs and checking if any of the input addresses have a payment credential that matches the `programmable_logic_base`.

Relevant Code Snippet

```
1 // Can we delegate to PLGlobal?
2 // PLGlobal is mint-aware – delegate all validation
3 // (handles both minting and burning accounting)
4 list.any(
5   self.inputs,
6   fn(input) {
7     input.output.address.payment_credential == programmable_logic_base
8   },
9 ),
```



Recommendation

Instead of scanning through all transaction inputs to check for a PLB input, it is more efficient to check for the presence of the PLGlobal delegation credential directly in the transaction's withdrawals. This provides a more direct signal of delegation to PLGlobal and reduces execution cost.

However, checking for the PLGlobal delegation credential in the withdrawals instead of the PLB input may allow for a scenario where the PLGlobal delegation credential is included in a transaction without including a PLB input. Therefore, it is important to ensure that no critical logic in the PLGlobal can be bypassed by not including a PLB input.

Note: Special attention should be given to the PLGlobal `ThirdPartyAct` path, which has some documented issues.

References

Finding **F501**: Escape of Minted Tokens during Third-Party Action

Resolution

The transaction-input scan was replaced by a policy-scoped PLGlobal withdrawal-redeemer signal tied to the same registry node. Missing, mismatched, or unsupported redeemers fall back to local `no_escape` custody validation, and `UnfrackingAct` cannot authorize issuance delegation.

[I 106] Redundant `single_mint_with_credential` Check in `issuance_mint`

Level	Severity	Status
 1	Informational	Resolved

Category: Optimization

Affected File: `validators/issuance_mint.ak`

Description

In the `issuance_mint` validator, the `single_mint_with_credential` function is used to ensure that there is no other minting redeemer in the transaction that matches the redeemer being used in the current validation.

Relevant Code Snippets

```

1  and {
2    // Validate that the redeemer matches the minting logic credential
3    (redeemer.minting_logic_cred == minting_logic_cred)?,
4    // substandard minting invoked?
5    list.has(invoked_scripts, minting_logic_cred)?,
6    // Check that exactly one minting redeemer matches the minting logic credential
7    // This prevents multiple concurrent mints with the same minting logic in one
8    // transaction
9    // What is this for?
10   single_mint_with_credential(self.redeemers, redeemer)?,
11   valid_minting?,
12 }

```



```

1  /// Check that exactly one minting redeemer with the given action exists in
2  the transaction
3  /// This prevents multiple concurrent mints with the same minting logic credential
4  ///
5  /// The logic follows Plutarch's pSingleMintWithCredential (lines 145-193):
6  /// 1. Scan all redeemers
7  /// 2. Filter for Mint purposes only
8  /// 3. Count how many match the given redeemer
9  /// 4. Ensure exactly one match exists
10 fn single_mint_with_credential(
11   redeemers: Pairs<ScriptPurpose, Data>,
12   target_redeemer: SmartTokenMintingAction,
13 ) -> Bool {
14   let target_redeemer_data: Data = target_redeemer

```



```

14
15  let matching_count =
16    list.foldl(
17      redeemers,
18      0,
19      fn(pair, count) {
20        let Pair(purpose, rdmr) = pair
21
22        when purpose is {
23          Mint(_) ->
24            if rdmr == target_redeemer_data {
25              count + 1
26            } else {
27              count
28            }
29          _ -> count
30        }
31      },
32    )
33
34  matching_count == 1
35 }

```

Here, the `single_mint_with_credential` function is scanning through all redeemers in the transaction, filtering for those with a `Mint()` purpose, and counting how many of those redeemers match the `target_redeemer`. Finally, it checks that there is exactly one matching redeemer.

Assessment

While this check may have been intended to prevent multiple concurrent mints with the same minting logic credential, it does not provide any clear security benefit that is not already provided by the structure of the minting logic. Additionally, there is no clear attack scenario that would be enabled by removing this check.

However, this check does add extra execution cost due to the need to scan through all redeemers and perform the counting logic. Therefore, it may be worth considering whether this check is necessary or if it can be removed to optimize execution cost without compromising security.

Recommendation

Given that the `single_mint_with_credential` function does not provide a clear security benefit and adds extra execution cost, it may be advisable to remove this check from the `issuance_mint` validator. This would simplify the validation logic and reduce execution cost without introducing any known vulnerabilities.

1. Remove the `single_mint_with_credential` function from the codebase.
2. Update the `issuance_mint` validator to no longer call `single_mint_with_credential`.
3. Remove `redeemer.minting_logic_cred == minting_logic_cred` check from the validation logic, as it is no longer necessary.
4. Remove the `minting_logic_cred` field from the redeemer, as it is no longer used.

These changes would streamline the validation process and reduce execution cost while maintaining the security of the minting logic.

Resolution

The transaction-wide matching-mint-redeemer scan, `single_mint_with_credential` helper, redundant redeemer credential field, and wrapper type were removed. Registry binding and the required minting-logic withdrawal preserve the necessary authorization relationship.

[I 107] Redundant Checks and Misleading Variable Names

Level	Severity	Status
1	Informational	Resolved

Category: Optimization / Code Quality

Affected Files:

- `validators/registry_mint.ak`
- `lib/linked_list.ak`

In the `RegistryInsert` branch of the `registry_mint` validator, there are several checks that are redundant under the current assumptions and helper definitions. These redundancies can be removed to simplify the code without affecting its correctness. Below are the identified redundancies and the rationale for their removal:

1. The singleton pattern:

In the `validate_directory_node_output` function, the check:

- `expect dict.size(node_tokens) == 1`

is redundant because the line `expect [Pair(tn, qty)] = dict.to_pairs(node_tokens)` already proves that `node_tokens` has exactly one token name. Therefore, the size check can be removed.

```

1  let node_tokens = tokens(value, node_cs)
2  - expect dict.size(node_tokens) == 1
3
4  expect node: RegistryNode = datum
5  expect bytearray_lt(node.key, node.next)
6  - let token_pairs = dict.to_pairs(node_tokens)
7  - expect [Pair(tn, qty)] = token_pairs
8  + expect [Pair(tn, qty)] = dict.to_pairs(node_tokens)
9  expect qty == 1

```

2. The exception for the origin node:

In the same function `validate_directory_node_output`, the check:

- `expect tn == node.key || tn == origin_node_tn`

is redundant because `origin_node_tn` is defined as `#" "`, and the `init` function already requires that the origin node's `node.key == #" "`. Therefore, we can simplify this check to just compare `tn` with `node.key`.

```

1  - expect tn == node.key || tn == origin_node_tn
2  + expect tn == node.key

```

Additionally, in the `RegistryInit` branch, in the `validate_directory_init` function, the check `expect node.key == #"` can be replaced by `expect node.key == origin_node_tn` for better readability and maintainability, as it uses the defined constant for the origin node token name (defined in `lib/registry_node.ak`).

```
1 - expect node.key == #""
2 + expect node.key == origin_node_tn
```

diff

This change secures the code against future modifications where the origin node token name might be changed, ensuring that all references to it are consistent.

3. The length check for the key:

In the main `RegistryInsert` branch, the check `(bytearray.length(key) == 28)?` is redundant because the `is_token_registered?` helper already requires that the computed policy ID matches the key, and the `apply_hashed_parameter` function returns a 28-byte policy ID. Therefore, if the computed policy ID matches the key, the key must already be 28 bytes long.

```
1 and {
2   is_token_registered?,
3 - (bytearray.length(key) == 28)?,
4   ...
5 }
```

diff

4. The ordering checks for the covering node:

In the same main `RegistryInsert` branch, the checks `bytearray_lt(covering_node.key, key)?` and `bytearray_lt(key, covering_node.next)?` are redundant. Since the `validate_directory_node_output` function already checks that `node.key < node.next`, and the helper definitions for `registry_node_updated?` and `registry_node_inserted?` enforce the relationships between the covering node and the new key, these checks can be removed.

```
1 and {
2   is_token_registered?,
3 - (bytearray.length(key) == 28)?,
4 - bytearray_lt(covering_node.key, key)?,
5 - bytearray_lt(key, covering_node.next)?,
6   just_single_mint?,
7   (list.length(output_nodes) == 2)?,
8   registry_node_updated?,
9   registry_node_inserted?,
10 }
```

diff

5. The `just_single_mint?` check:

In the `RegistryInsert` branch, the `just_single_mint?` check is technically redundant under the following stronger assumptions:

```
1 expect [covering_input] = node_inputs
2 expect [Pair(tn, qty)] = dict.to_pairs(node_tokens)
3 expect qty == 1
4 expect tn == node.key
5 list.length(output_nodes) == 2
6 registry_node_inserted?
7 registry_node_updated?
```

The above checks imply that the inserted output must carry one registry NFT whose token name is `key`. Since the covering input did not have token name `key`, the ledger balance requires the transaction to mint that NFT. However, since `just_single_mint?` is cheap, clear, and provides a direct check for the minting of the node token with the new key, it may be worth keeping for clarity and local reasoning about the minting behavior.

6. Computational redundancy in output node validation:

Currently, the code builds all output nodes, scans them twice with `list.any`, and separately checks their length. This can be refactored to validate the output nodes in a single pass while checking their order.

Replace the following code:

```
1 - // Parse the outputs to find the two new nodes
2 - let output_nodes =
3 -   list.map(
4 -     node_outputs,
5 -     fn(out) { validate_directory_node_output(out, policy_id) },
6 -   )
7 -
8 - // One output must be the updated covering node (key -> insert_key)
9 - let registry_node_updated =
10 -   list.any(
11 -     output_nodes,
12 -     fn(node) { is_updated_directory_node(node, covering_node, key) },
13 -   )
14 -
15 - // One output must be the new inserted node (insert_key -> next)
16 - let registry_node_inserted =
17 -   list.any(
```

```

18 -         output_nodes,
19 -         fn(node) {
20 -             is_inserted_directory_node(node, key, covering_node.next)
21 -         },
22 -     )
23 +     // Exactly two registry node outputs are expected:
24 +     // one updated covering node and one newly inserted node.
25 +     expect [node_output_a, node_output_b] = node_outputs
26 +
27 +     let node_a =
28 +         validate_directory_node_output(node_output_a, policy_id)
29 +
30 +     let node_b =
31 +         validate_directory_node_output(node_output_b, policy_id)
32 +
33 +     // Outputs may appear in either order. (Alternatively, we could require a
34 +     // specific order and simplify this check.)
35 +     let registry_nodes_valid =
36 +         if node_a.key == covering_node.key {
37 +             and {
38 +                 is_updated_directory_node(node_a, covering_node, key)?,
39 +                 is_inserted_directory_node(node_b, key, covering_node.next)?,
40 +             }
41 +         } else {
42 +             and {
43 +                 is_inserted_directory_node(node_a, key, covering_node.next)?,
44 +                 is_updated_directory_node(node_b, covering_node, key)?,
45 +             }
46 +         }
47 +
48 -         // Must have exactly 2 outputs: the updated covering node and the new
49 -         inserted node
50 -         (list.length(output_nodes) == 2)?,
51 -         registry_node_updated?,
52 -         registry_node_inserted?,
53 +         registry_nodes_valid?,

```

This refactor removes the separate length check and the two `list.any` traversals while preserving the behavior that the outputs may appear in either order. It also makes the validation logic more direct and easier to follow.

7. Misleading naming of variables:

In the `RegistryInsert` minting path there are several names that are misleading enough to make review harder, or cause confusion about the code's intent:

- `is_token_registered`: This name suggests that the function checks if a token is registered in the registry, but it actually checks if the key is a valid official programmable-token registration candidate by verifying that the computed policy ID matches the key and that the policy is minting in this transaction. A more accurate name could be `is_valid_registration`, `valid_registration`, or `is_official_issuance_policy_mint`.
- `just_single_mint`: This name suggests that the entire minting action involves exactly one token, but it only checks that the registry NFT for the key is minted under the registry policy. A clearer name could be `inserted_node_nft_minted`, `new_registry_nft_minted`, or `has_insert_key_nft_mint`.

Resolution

The redundant singleton, key-length, ordering, and multi-pass output-classification checks were removed, misleading names were eliminated, and origin handling now uses strict NFT-name-to-node-key equality plus canonical origin-datum validation. The direct registry-NFT mint assertion was intentionally retained for clear local reasoning.

[I 108] ThirdPartyAct Cannot Atomically Operate Across Multiple Token Policies

Level	Severity	Status
 1	Informational	Acknowledged

Category: Design Limitation

Affected Files:

- `validators/programmable_logic_global.ak`
- `validators/programmable_logic/third_party.ak`

Description

`validate_3rd_party` resolves exactly one `registry_node_idx` to one `policy_id` per invocation. Because a withdrawal script can only be invoked once per transaction, only one token policy can be processed in a single `ThirdPartyAct` transaction. This prevents any compliance operation that must atomically involve tokens from two or more distinct policies.

Scenario

Token conversion / replacement

A compliance operator needs to atomically retire two legacy token policies (A and B) and issue a replacement token (C), such as during a token migration, a corporate restructuring, or a regulatory-mandated instrument replacement.

- Burning policy A tokens and policy B tokens must occur in the same transaction as minting policy C tokens to preserve atomicity and auditability.
- Under the current design, only one of A or B can be processed per `ThirdPartyAct` invocation, making a true atomic swap impossible.
- The operator must accept a multi-step process with an exposure window between steps, or rely on off-chain coordination to approximate atomicity.

Impact

Operations involving more than one token policy require multiple sequential transactions rather than a single atomic one, adding operational steps and leaving intermediate states briefly visible on-chain.

Recommendation

Extend the redeemer to accept a list of policy references:

```
1 ThirdPartyAct { actions: List<(registry_node_idx: Int, outputs_start_idx: Int)> }
```

 Aiken

Each entry targets one policy; all are validated in a single transaction, bounded only by execution-unit and size limits. This enables atomic multi-policy compliance operations without a protocol redesign.

Resolution

`ThirdPartyAct` remains limited to one policy per invocation, so multi-policy administrative operations require sequential transactions and retain an exposure window. The client explicitly documented and accepted this design tradeoff, and no equivalent atomic compensating path was implemented.

[I 109] Token UTxO Fragmentation Causes Non-Atomic Seizure

	Level	Severity	Status
	1	Informational	Resolved

Category: Design Limitation

Affected File: `validators/programmable_logic/third_party.ak`

Description

A token holder can spread their balance across many UTxOs. Because `check_seized_tokens` pairs PLB inputs with outputs positionally and Cardano transactions are bounded by size and execution-unit limits, only a limited number of UTxOs can be processed per transaction. When a compliance operator needs to seize the holder's entire balance, too many UTxOs may exceed what fits in a single transaction, requiring multiple transactions to complete the full seizure.

Scenario

Complete seizure of a highly fragmented holder

- A holder with more UTxOs than fit in a single transaction requires multiple seizure transactions to seize the full balance, these transactions may be included in the same block or across separate blocks.
- If the transactions span multiple blocks, unprocessed UTxOs remain live and spendable between blocks.
- Each transaction may succeed or fail independently, making partial completion a realistic outcome.

Impact

When seizing all tokens from a highly fragmented holder, the operation may be only partially successful if not all transactions land in the same block. Some UTxOs may be seized while others remain accessible to the holder during the multi-transaction window. This weakens compliance guarantees compared to an atomic, all-or-nothing operation.

If the token requires atomic full seizure, a freeze mechanism must be in place as a prerequisite.

Recommendation

Document this limitation explicitly in the CIP-113 specification so that compliance operators are aware of the multi-transaction nature of seizure for fragmented holders.

Third-party substandards that require strong seizure guarantees should include a freeze mechanism as a prerequisite step. Freezing must block all spends from the target's credentials before any seizure transaction is submitted, eliminating the window in which remaining UTxOs can be moved. The substandard specification should state this as a required guarantee rather than leaving it as an implementation detail.

Resolution

`documentation/03-CONTROL-SCOPE-AND-ADMIN-AUTHORITY.md` §3.4 now explicitly documents that `ThirdPartyAct` affects only the UTXOs included in a transaction, fragmentation can exceed transaction-size or execution-budget limits, and full seizure may therefore require multiple transactions with an exposure window. §§2.3–2.5 and `documentation/09-DEVELOPING-SUBSTANDARDS.md` assign token-specific freeze and seizure policy to each substandard’s transfer and third-party logic. These additions resolve the informational design-limitation finding.

[I 110] ThirdPartyAct Scope is Not Clearly Defined

Level	Severity	Status
 1	Informational	Partially Resolved

Category: Administrative Control / Transparency

Affected File: `validators/programmable_logic_global.ak`

Description

In the current implementation of the `ThirdPartyAct` path in the PLGlobal, there is a lack of clarity regarding the scope of actions that an admin can perform.

The current implementation of the `ThirdPartyAct` path allows for a wide range of actions that an admin can perform, without clear guidelines or restrictions. This can lead to potential misuse of the admin privileges, or unintended consequences for users interacting with the protocol.

Currently, the `ThirdPartyAct` gives the admin control over all UTxOs that holds the administered token, including the UTxOs that belong to users, DeFi protocols, DEXs, and other applications. This means that the admin can perform actions on these UTxOs, such as consuming them, which can have significant implications for users and the overall ecosystem.

Impact

One of the main impacts of this issue is related to the usability and security of DeFi protocols. While the `ThirdPartyAct` path is intended for administrative purposes, it can be used to perform actions that may be considered “dangerous” or beyond the scope of typical administrative functions. For example, in DeFi applications, the `ThirdPartyAct` path can be used to consume collateral tokens, or may be able to force liquidation of positions, which can have significant financial implications for users and may affect the usability and security of DeFi protocols.

Users that want to interact with DeFi applications may be hesitant to do so if they know that an admin has the power to perform actions that could negatively impact their assets or positions. This can lead to a lack of trust in the protocol and may discourage users from participating in the DeFi ecosystem. For example, if an admin can seize collateral tokens, a lender may be hesitant to lend assets on the platform, as they may fear that the collateral that covers their loan could be seized by the admin. This can lead to a decrease in liquidity and overall participation in the DeFi protocol, which can ultimately harm the growth and success of both the DeFi protocol and the CIP-113 standard.

Recommendation

1. Clearly define the scope of actions that the admin can perform through the `ThirdPartyAct` path, and ensure that these actions are limited to necessary administrative functions. This will help to prevent misuse of admin privileges and reduce the risk of unintended consequences for protocols implementing the CIP-113 standard.

2. After clearly defining the scope of admin actions, document these actions in the protocol's documentation to provide transparency to users and build trust in the protocol. This will help users understand the limitations of admin actions and make informed decisions when interacting with the protocol.
3. Update the protocol's implementation to match the defined scope of admin actions, ensuring that any action that is considered "dangerous" or beyond typical administrative functions is not allowed through the `ThirdPartyAct` path. This will help to ensure that the protocol's implementation aligns with the defined scope of admin actions and reduces the risk of misuse of admin privileges. This may involve implementing additional checks or restrictions on the actions that can be performed through the `ThirdPartyAct` path, to ensure that they are limited to necessary administrative functions and do not pose a risk to users or the overall ecosystem.

Resolution

`documentation/03-CONTROL-SCOPE-AND-ADMIN-AUTHORITY.md` §§2.1 and 2.4–2.5 define the `ThirdPartyAct` guarantees, disclose its reach over script-held DeFi positions, and assign holder-scope restrictions to substandard allowlist or consent logic. `documentation/09-DEVELOPING-SUBSTANDARDS.md` provides the corresponding implementation guidance, `documentation/02-ARCHITECTURE.md` summarizes the administrative flow, and `documentation/08-INTEGRATION-GUIDES.md` distinguishes administrative actions from owner-authorized transfers. These additions provide the requested scope documentation, but the final documentation still describes an asset-name exemption absent from the implementation, and the base path permits an authorized administrator to act on a DeFi UTXO unless the registered substandard imposes the documented restriction.

[I 111] Registry Design Limitation

Level	Severity	Status
1	Informational	Partially Resolved

Category: Design Limitation

Affected Files:

- `validators/registry_spend.ak`
- `validators/registry_mint.ak`
- `lib/registry_node.ak`

Description

The current registry design does not allow for updating registry nodes or de-registering them. This means that once a registry node is added to the list, it cannot be removed or updated.

The current registry node structure is as follows:

```

1  pub type RegistryNode {
2      /// The key (currency symbol) of the programmable token policy
3      key: ByteArray,
4      /// The next key in lexicographical order (for linked list traversal)
5      next: ByteArray,
6      /// The transfer logic script credential that validates token transfers
7      transfer_logic_script: Credential,
8      /// The third party transfer logic script credential that handles admin actions
9      third_party_transfer_logic_script: Credential,
10     /// Optional global state NFT currency symbol for this policy
11     global_state_cs: ByteArray,
12 }

```

Aiken

The `transfer_logic_script` and `third_party_transfer_logic_script` fields are crucial for the functionality of the registry nodes, as they define the logic for validating token transfers and handling admin actions. If there are bugs or vulnerabilities in the initial implementation of these scripts, there is currently no way to update them, which could lead to potential security issues.

Current Limitations

1. No way to fix issues: If there are bugs or vulnerabilities in the initial implementation of the transfer logic script or the third-party transfer logic script, there is currently no way to update them, which could lead to potential security issues.
2. No way to add features: If the issuer wants to add new features to the transfer logic or the third-party transfer logic, there is currently no way to do that, which could limit the functionality of the registry nodes.

3. No way to de-register tokens and move out of the CIP-113: If a stable-coin issuer decides that they want to move out of this cip-113 compliant registry, there is currently no way to de-register their token from the registry.

In the current design, if a token needs to be updated, there is no direct way to do that, instead, the issuer would have to create a new token, register it, and then migrate all the users to the new token, which could be a complex and error-prone process. Additionally, the old token would still be in the registry, and its minting logic would still be active, which could lead to confusion and potential security issues.

Recommendations

The main question is who should have the authority to perform registry-related actions, such as registering a programmable token, updating the token's registry node, and de-registering the token. One potential solution is to use the `issuanceLogicScript` as the main component in authorizing registry-related actions.

We recommend defining a `RegistryLifecycleAuthority` witness, which proves that the token-specific issuance authority has authorized a registry lifecycle action (register, update, de-register).

The `RegistryLifecycleAuthority` can be defined as follows:

For a programmable token policy `P`, `RegistryLifecycleAuthority` means:

1. `P` is the official `issuanceMintingPolicy` instance, derived from the provided `issuanceLogicScript`.
2. The corresponding `issuanceLogicScript` is executed through `withdraw-zero`.
3. `tx.mint` has no entries under policy `P`.

Note: the 3rd condition (`tx.mint` has no entries under policy `P`) is used to separate the authority of the `RegistryLifecycleAuthority` from the authority of issuing new tokens, as the `RegistryLifecycleAuthority` should only have the authority to perform registry-related actions, and not the authority to mint new tokens. Similarly, issuing new tokens should not grant the authority to perform registry-related actions.

The `RegistryLifecycleAuthority` can be implemented as follows (conceptually):

Implementation note: If `issuanceLogicScript` is reused as the registry lifecycle authority, the registry must not merely check that some withdrawal script was executed. It must prove that the executed `issuanceLogicScript` is the same credential that parameterizes the official `issuanceMintingPolicy` instance for policy `P`. The exact parameter bytes used to reconstruct the issuance policy hash are serialization-sensitive and should be confirmed with golden tests against the compiled/applied issuance policy.

```
1 fn credential_to_bytes(cred: Credential) -> ByteArray {
2   builtin.serialise_data(as_data(cred))
```

 Aiken

```

3  }
4
5  pub fn has_registry_lifecycle_authority(
6    token_policy: ByteArray,
7    issuance_logic_cred: Credential,
8    all_mint_entries: Value,
9    withdrawals: Pairs<Credential, Lovelace>,
10   prefix: ByteArray,
11   postfix: ByteArray,
12  ) -> Bool {
13    let param_bytes = credential_to_bytes(issuance_logic_cred)
14    let computed_policy = apply_hashed_parameter(prefix, postfix, param_bytes)
15
16    and {
17      // Check if the issuanceLogicScript is executed via withdraw-zero
18      list.any(
19        withdrawals,
20        fn(wdrl) -> Bool {
21          let Pair(cred, _amount) = wdrl
22          cred == issuance_logic_cred
23        },
24      ),
25      // Check if tx.mint has no entries under policy P
26      dict.is_empty/assets.tokens(all_mint_entries, token_policy)),
27      // Check if the issuanceMintingPolicy instance is derived from the provided
28      // issuanceLogicScript
29      computed_policy == token_policy,
30    }
31  }

```

The `RegistryLifecycleAuthority` can then be used to authorize the following registry-related actions:

1. Register a programmable token: The `RegistryLifecycleAuthority` can be used to authorize the registration of a new programmable token in the registry.
2. Update the token's registry node: The `RegistryLifecycleAuthority` can be used to authorize updates to the transfer logic script, third-party transfer logic script, or global state of an existing registry node.
3. De-register the token: The `RegistryLifecycleAuthority` can be used to authorize the de-registration of a token from the registry.

- Note that the de-registration path should not allow for the complete deletion of the registry node, as this would lead to the following potential security issue:
 - If a registry node is deleted (completely removed from the registry), it could potentially be re-registered, which would create an escape window for the tokens to be moved outside the PLB.
 - Attack scenario: The token issuer de-registers their token from the registry, then tokens are transferred or minted outside the PLB, and then the issuer re-registers the same token back to the registry. This would allow the tokens to effectively escape the PLB, which would create a major security issue.
- Instead, the de-registration path should clearly mark the registry node as de-registered, which would prevent re-registration and ensure that the token is effectively removed from the registry while still maintaining a record of its existence. This could be done by adding a “de-registration” flag in the registry node, which would indicate that the node is no longer active and should not be used for any registry-related actions. Note also that the de-registration path should be irreversible, meaning that once a registry node is marked as de-registered, it cannot be re-activated or re-registered.

Summary

The current registry design does not allow for updating registry nodes or de-registering them, which could lead to potential security issues and limit the functionality of the registry. We recommend defining a `RegistryLifecycleAuthority` that would have the authority to perform registry-related actions, such as registering a programmable token, updating the token’s registry node, and de-registering the token. This would allow for a more flexible and secure registry design, as it would allow for updating registry nodes and de-registering them when necessary.

Resolution

Registry nodes now have an authorized in-place update path that preserves identity, position, custody, and issuance bindings. Irreversible de-registration remains absent: there is no inactive or tombstone state, and every reachable lifecycle spend preserves an active node.

Appendix A: Supporting Validation Matrices

The following matrices summarize supporting validation analysis performed during the assessment.

They are provided as auxiliary artefacts to clarify transaction-path behavior, expected versus observed outcomes, and ThirdPartyAct use-case coverage.

They should be read together with the corresponding findings and not as a substitute for the detailed finding descriptions.

A.1 Registration, Minting, and Transfer Matrix

Case #	Registering	Minting	Transferring	Expected	Actual	Result	Notes
1	Token A	No	No	Fail	Fail	Pass	Registering a token requires minting it
2	Token A	Token A	No	Succeed	Succeed	Pass	
3	Token A (re-reg)	Token A	No	Fail	Fail	Pass	Re-registering an already registered token would fail
4	Token A	Tokens A & A-1	No	Succeed	Succeed	Pass	
5	Token A	Token A	TransferAct A	Fail	Fail	Pass	Impossible (Token A do not exist before registration)
6	Token A	Token A	ThirdPartyAct A	Fail	Fail	Pass	Impossible (Token A do not exist before registration)
7	Token A	Tokens A	TransferAct B	Succeed	Fail	Fail	PLG's 'choose_registered_token_with' can't prove A
8	Token A	Tokens A	ThirdPartyAct B	Succeed	Succeed	Pass	
9	Token A	Tokens A	TransferAct (A-1)	Succeed	Fail	Fail	This is a known limitation of on-chain linked lists
10	Token A	Tokens A	ThirdPartyAct (A-1)	Succeed	Fail	Fail	This is a known limitation of on-chain linked lists
11	Token A	Tokens A & B	No	Succeed	Succeed	Pass	
12	Token A	Tokens A & B	TransferAct B	Succeed	Fail	Fail	PLG's 'choose_registered_token_with' can't prove A
13	Token A	Tokens A & B	ThirdPartyAct B	Succeed	Succeed	Pass	
14	Tokens A & B	Any	Any	Fail	Fail	Pass	Requires exactly one node input and exactly two node outputs
15	No	Token B	No	Succeed	Succeed	Pass	
16	No	Token B	Any B	Succeed	Succeed	Pass	
17	No	Token B	TransferAct C	Succeed	Succeed	Pass	
18	No	Token B	ThirdPartyAct C	Succeed	Vulnerable	Fail	Minted token B can escape CIP-113's smart-wallet (PLB)
19	No	Tokens B	ThirdPartyAct B, C	Succeed	Succeed	Pass	same address, same datum, same non-B value, B value must change
20	No	Tokens B & C	No	Succeed	Succeed	Pass	
21	No	Tokens B & C	TransferAct B	Succeed	Succeed	Pass	
22	No	Tokens B & C	ThirdPartyAct B	Succeed	Vulnerable	Fail	Minted token C can escape CIP-113's smart-wallet (PLB)
23	No	Tokens B & C	TransferAct B & C	Succeed	Succeed	Pass	
24	No	Tokens B & C	ThirdPartyAct B, C	Succeed	Vulnerable	Fail	Minted token C can escape CIP-113's smart-wallet (PLB)
25	No	No	Any	Succeed	Succeed	Pass	Any valid TransferAct/ThirdPartyAct over registered tokens

Notes:

- B and C are already registered programmable tokens; A is not registered.
- A-1 means the covering/predecessor node consumed during RegistryInsert(A).
- TransferAct rows assume the redeemer includes complete proofs for all tx.mint policies and all consumed PLB-input policies.
- ThirdPartyAct B, C means selected policy B; C may be present in PLB inputs only as an unchanged non-B value.

Note: In this matrix, “Pass” and “Fail” describe whether the observed behavior matched the assessed expectation for that specific scenario. They do not indicate remediation status. Rows marked “Fail” identify behavior mismatches, limitations, or vulnerable cases that are discussed in the corresponding findings.

A.2 ThirdPartyAct Use-Case Coverage Matrix

Use case	Example	Actual	Status	Notes
Seizure	Admin removes frozen/illegal funds	Possible	Pass	
Forced transfer	Move user funds to legal recipient/escrow	Possible	Pass	
Wipe / burn	Destroy seized tokens	Possible	Pass	
Top-up / mint	Add tokens during admin action	Partial	Pass	At least one token needs to be present before this action. For security reasons, this is acceptable.
Corporate action / redenomination	2-for-1 split, 3-for-1 split, reverse split	Possible	Pass	
Recovery / migration	Move from compromised credential to new wallet	Partial	Limited	Only programmable tokens can be recovered. Non-programmable tokens (including ADA) are not recoverable with this action.
Regulatory clawback / fraud reversal	Reverse fraudulent/mistaken transfer	Partial	Limited	Only programmable tokens can be recovered. Non-programmable tokens (including ADA) are not recoverable with this action.
Escrow / quarantine	Move suspicious funds to quarantine wallet	Possible	Pass	
Freeze / unfreeze	Mark account or token frozen	Not Possible	Pass	Can be achieved by the 'TransferAct' Path (sub-standard implementation)
Pause / compliance update	Update global/user state	Not Possible	Pass	Can be achieved by the 'TransferAct' Path (sub-standard implementation)
Penalty / slashing / fee collection	Reduce user balance and send/burn penalty	Possible	Pass	
Rebase / haircut	Reduce/increase balances after supply event or loss event	Possible	Pass	
Vesting clawback	Reclaim unvested shares/tokens	Possible	Pass	
Share repurchase / cancellation	Company buys back or cancels restricted shares	Possible	Pass	
Forced redemption / expiry	Burn expired claim tokens or redeemable tokens	Possible	Pass	
Same-policy token conversion	Burn A.a + A.b, mint/give A.c	Possible	Pass	
Multi-policy token conversion	Burn A + B, mint/give C	Not Possible	Fail	Check finding 108
Liquidation	Protocol/admin moves collateral	Possible	Fail	ThirdPartyAct use case / integration risk? Check finding 110
Capital controls / transfer limits	Daily/monthly withdrawal limits, jurisdiction limits, blocking transfers outside allowed region	Not Possible	Pass	Can be achieved by the 'TransferAct' Path (sub-standard implementation)
Capital-control enforcement	Admin forcibly quarantines, reverses, or seizes tokens that violated controls	Partial	Limited	Only programmable tokens

Note: In this matrix, “Pass”, “Fail”, and “Limited” describe whether the assessed ThirdPartyAct behavior matches the expected capability for that use case. They do not indicate remediation status. “Pass” also does not imply that every token-specific substandard or per-policy script is safe.

End of Report



No Witness Labs — Confidential

Copyright © 2026